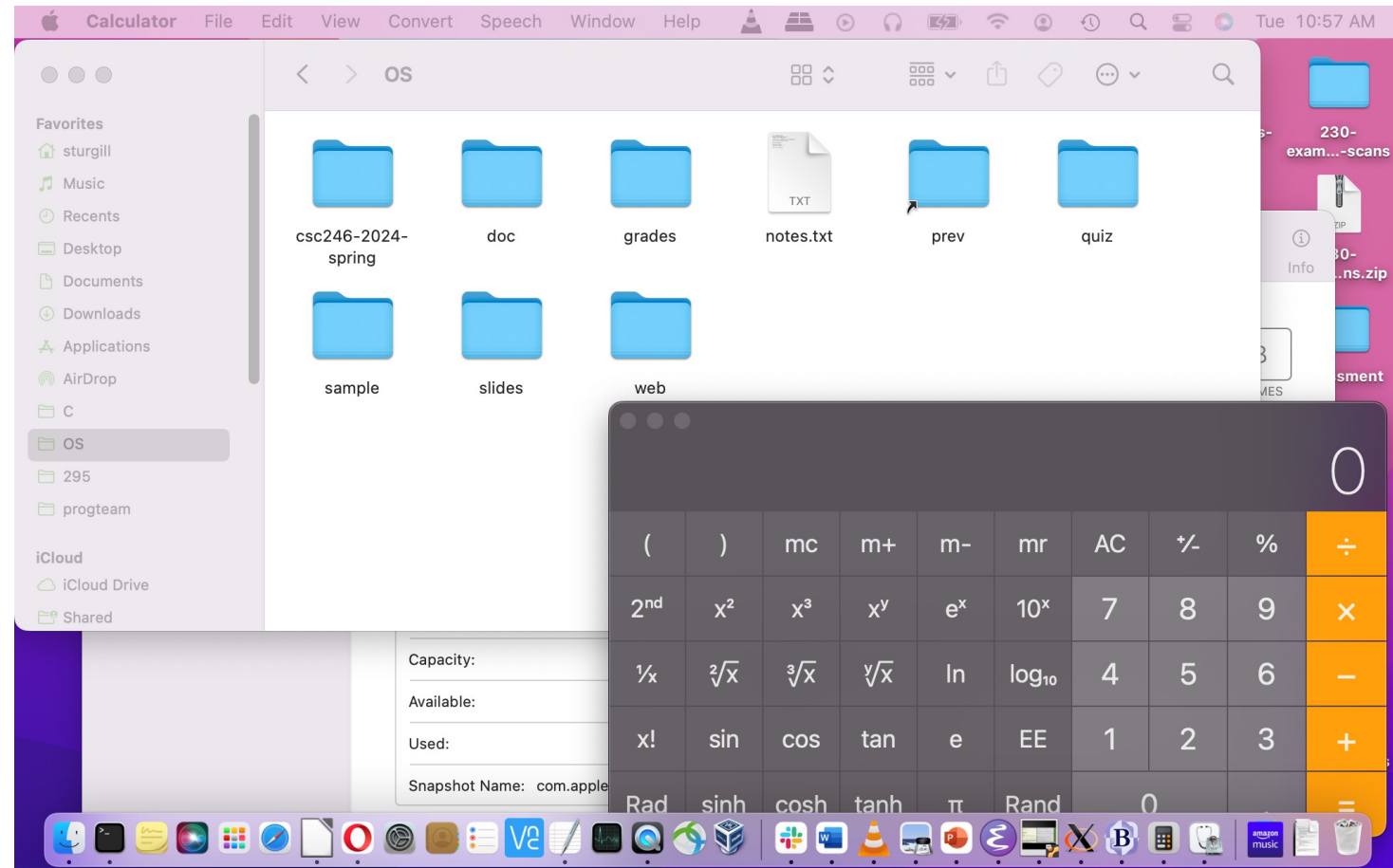


Working in Shell

CSC 230

Working in Shell

- Modern systems typically provide a graphical user interface.
 - For running programs, managing files, etc.
 - This is fine ... for some tasks



Working in Shell

- Often, systems will also provide a command-line interface.
 - This is better for many tasks.
- On a Unix machine, the command-interpreter is called the **shell**.
- On the common platform, the default shell is **bash** (the bourne again shell)

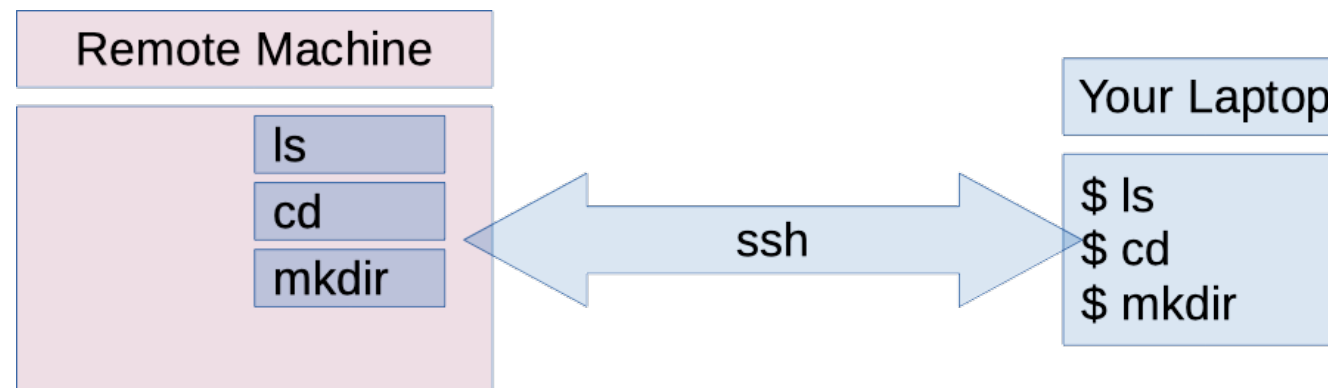
```
doc - dbsturgi@engr-ras-207:~/tmp -- bash -- 83x23
(/usr/local/texlive/2018basic/texmf-dist/tex/generic/oberdiek/ifpdf.sty))
(/usr/local/texlive/2018basic/texmf-dist/tex/latex/latexconfig/epstopdf-sys.cfg
))
Overfull \hbox (6.79999pt too wide) in paragraph at lines 14--23
[]
[1{/usr/local/texlive/2018basic/texmf-

tmp - bash - 80x24
Public/          fun/          sympathy-summary.ods
Video/           home/         talk/
VirtualBox VMs/ icpc_problem_notes.txt test2/
admin/           ideas.txt     tmp/
anti.zip         import/       trace.txt
app/             linguine.zip  trans/
back/            music-hash.txt utils.tgz
<castor 20> vi class/OS/notes.txt
<castor 21> cd /tmp
<castor 22> cd ~/tmp
<castor 23> gcc -Wall -std=c99 -Wall test.c -o test
test.c:5:1: warning: type specifier missing, defaults to 'int' [-Wimplicit-int]
05(02,07,03)char**07;{return!(02+==~01+01)?00:!(02==02>01)?printf("\045\157\012"
^
test.c:3:12: note: expanded from macro '05'
#define 05 main
^
test.c:5:1: error: third parameter of 'main' (environment) must be of type 'char
**'
test.c:3:12: note: expanded from macro '05'
#define 05 main
^
1 warning and 1 error generated.
<castor 24>

shell - bash
sandbox/garbage
sandbox/garbage/empty.txt
sandbox/garbage/trash.bin
sandbox/garbage/halla.c
sandbox/garbage/nonsense.txt
sandbox/readme.txt
sandbox/src
sandbox/src/random.c
sandbox/src/even.c
<castor 8> tar -czf sandbox.tgz sandbox
<castor 9> ncftp
[(dbsturgi@remote.eos.ncsu.edu) Password:
[(dbsturgi@remote.eos.ncsu.edu) Password:
Connected to remote.eos.ncsu.edu.
sftp> cd class/C/share
sftp> ls
ex    proj
sftp> put sandbox.tgz
Uploading sandbox.tgz to /mnt/coe/workspace/csc/CSC230/sandbox.tgz
sandbox.tgz          100% 3777      1.0MB/s   00:00
sftp> ^D
<castor 10> ls
doc/          sandbox/      sandbox.tgz   script.txt    script.txt~
<castor 11>
```

Working in Shell

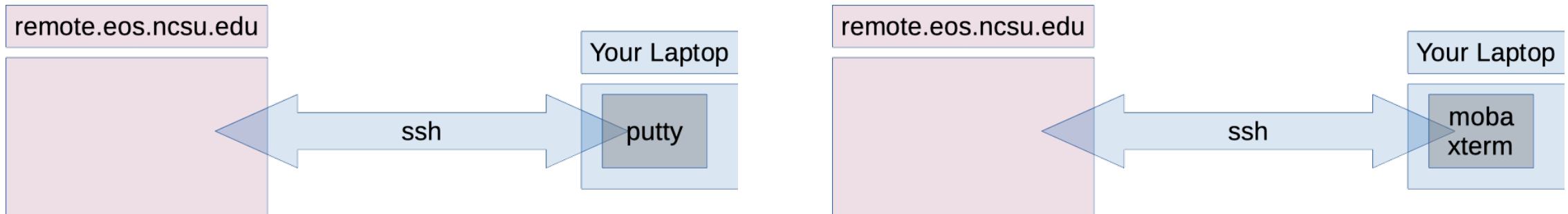
- Working in shell can be convenient
 - There's syntax for variables, looping, pattern matching, etc.
 - Shell is really a little programming language.
 - You can define shell functions or save a shell program in a file.
 - Using secure shell (ssh), you can use a shell on a remote machine.



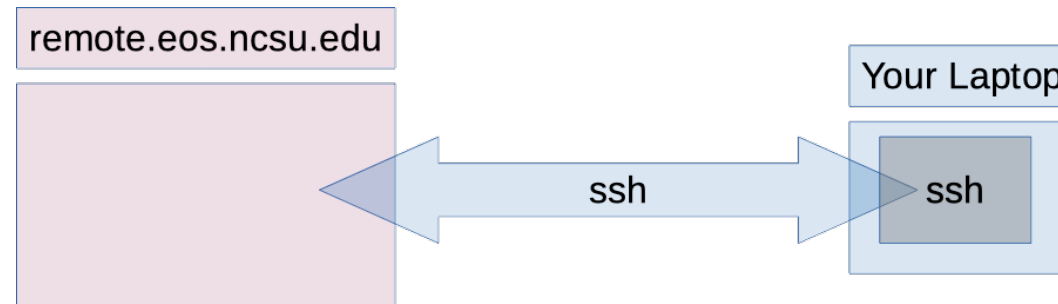
- NC State has a pool of Linux machines for remote access:
remote.eos.ncsu.edu

ssh clients

- There are lots of programs that work as ssh clients.



- MacOS and Windows now have a **ssh** command installed automatically.



Following Along



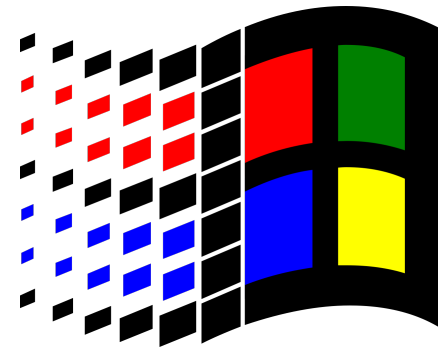
- You can work on one of the NCSU remote Linux machines.
- **On MacOS**
 - You can use the ssh client is already installed
 - Open a terminal window from Applications → Utilities → Terminal

Let's pretend this is
your terminal prompt.

Use your actual
unity ID.

Enter your NC State
password.

```
$ ssh unity-id@remote.eos.ncsu.edu  
(unity-id@remote.eos.ncsu.edu) Password:
```



Following Along

- You can work on one of the NCSU remote Linux machines.
- **On Windows 10 or 11**
 - A ssh client is already installed
 - Open a command window as: start menu → Command Prompt

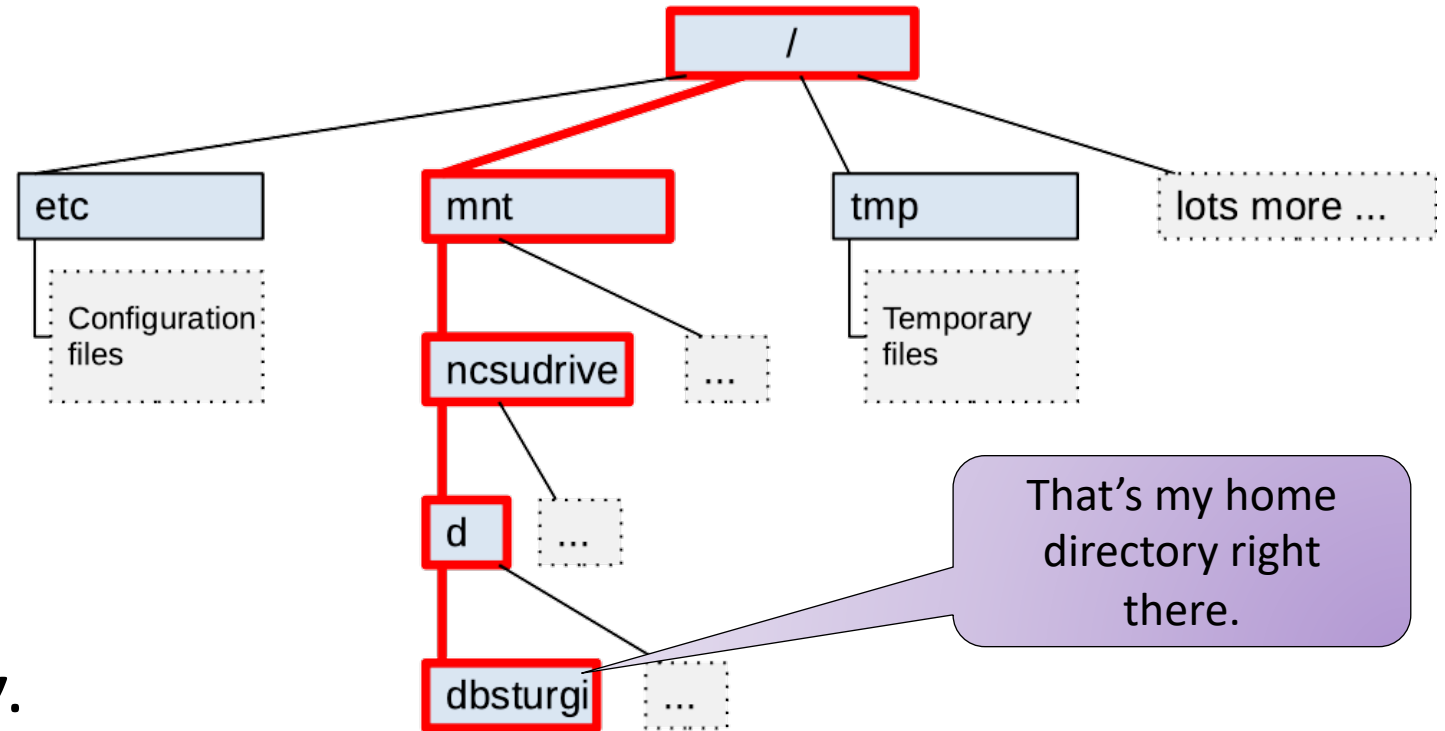
Use your actual
unity ID.

```
C:\Users\username>ssh unity-id@remote.eos.ncsu.edu  
Password:
```

Enter your NC State
password.

Directory Structure

- Unix systems have a mostly tree-shaped directory structure.
- I'll use a picture like this to show parts of it.
- When you log in, you start out in your ***home directory***.
 - There are lots of other files and directories.
 - Your home directory is the part of the space that's yours.



pwd : checking your current directory

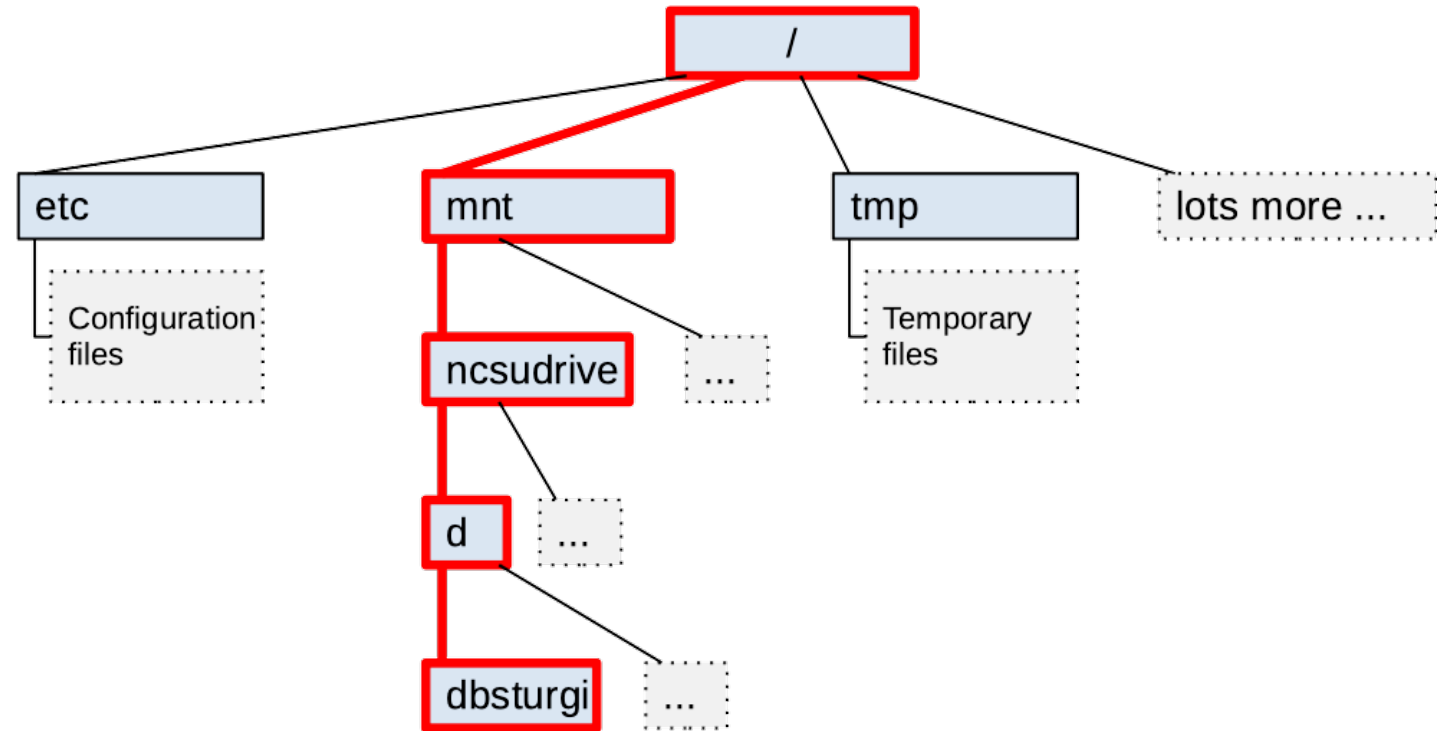
- The **pwd** command will report where you are in the filesystem.
- This is called your ***current directory***

We'll pretend this is your shell prompt (it's probably longer than that).

```
$ pwd
```

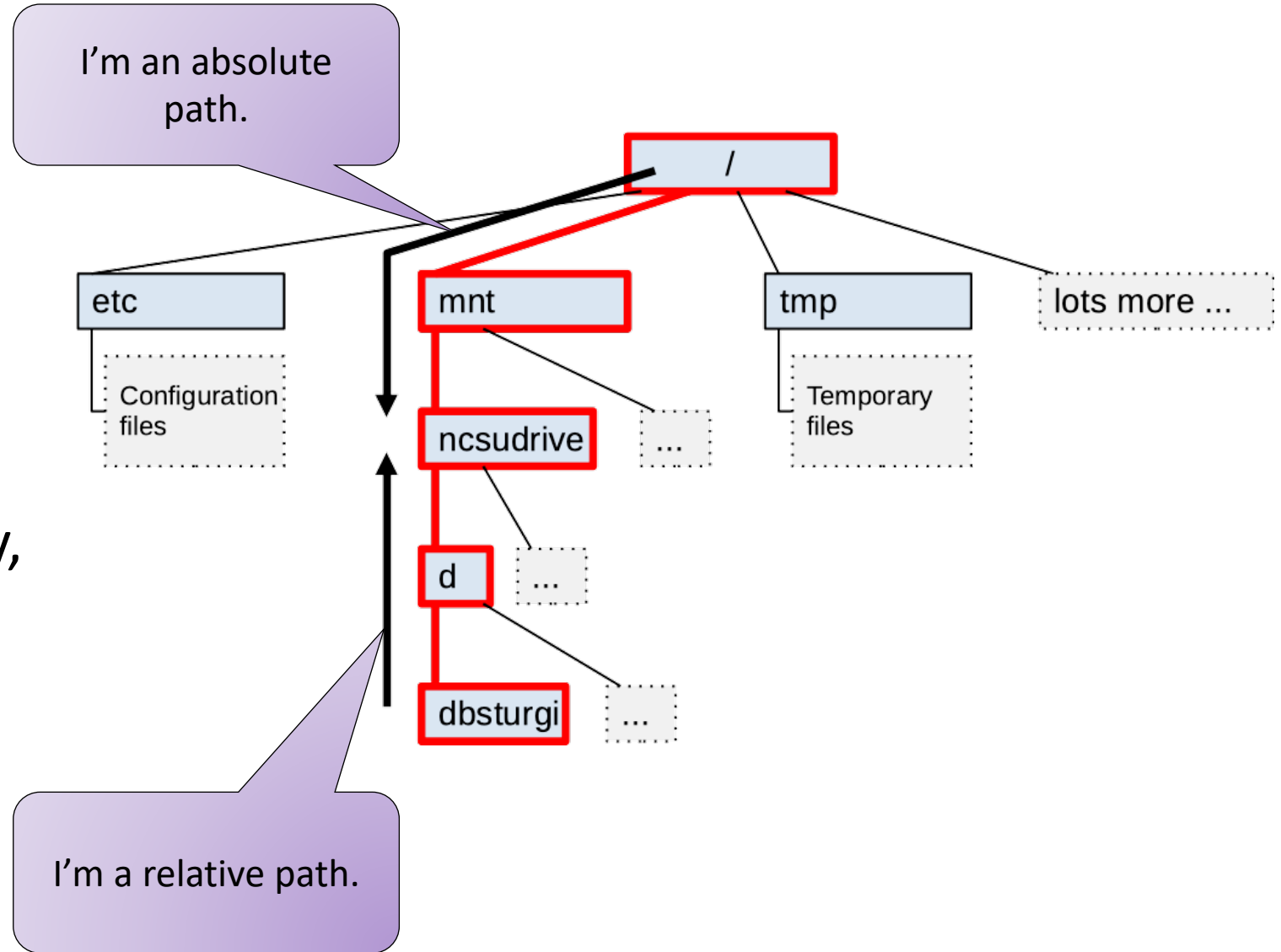
```
/mnt/ncsudrive/d/dbsturgi
```

Your directory will be different from the one I get.



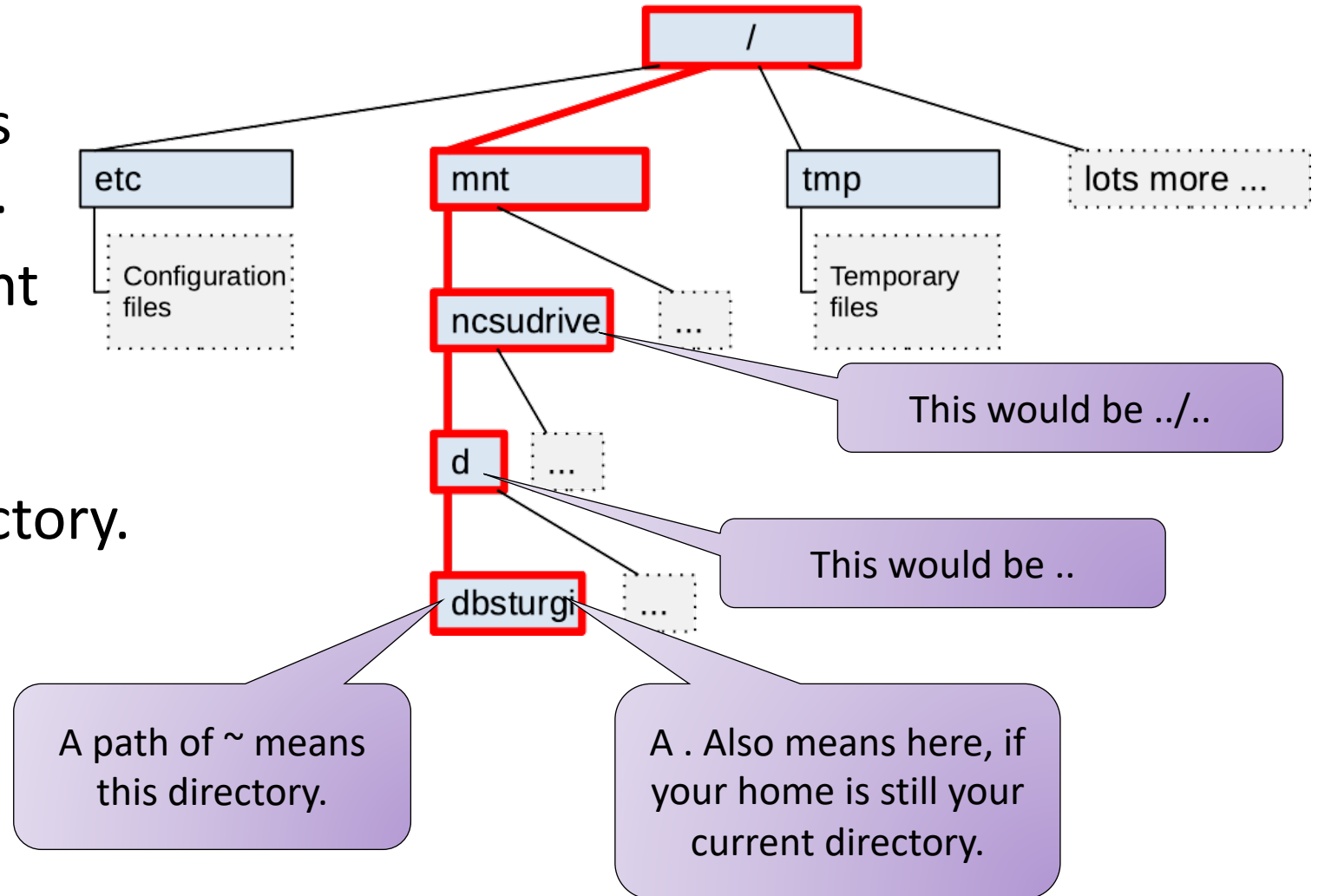
Paths

- A **path** is a name for a file or directory in the file system.
- An **absolute path** starts at the top-level directory, slash (/)
- A **relative path** starts at the current directory



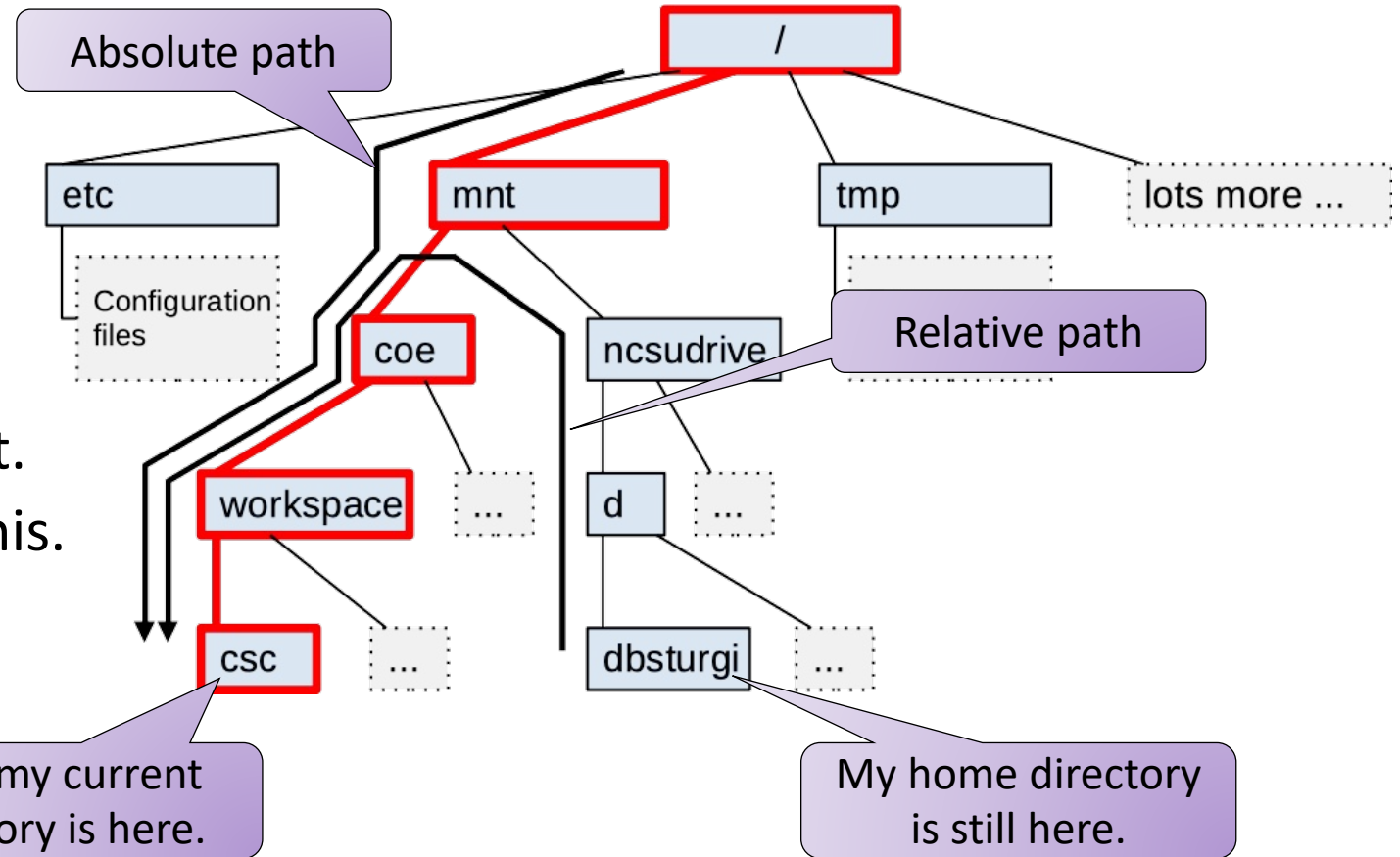
Path Name Syntax

- In a path, a dot (.) stands for the current directory.
- A dot dot (..) is the parent directory.
- For bash, a tilde (~) is an alias for your home directory.



cd : changing your current directory

- The **cd** command will let you move to a different directory.
 - It takes an absolute or relative path as an argument.
 - Lots of commands are like this.
- Lets go to `/mnt/coe/workspace/csc`

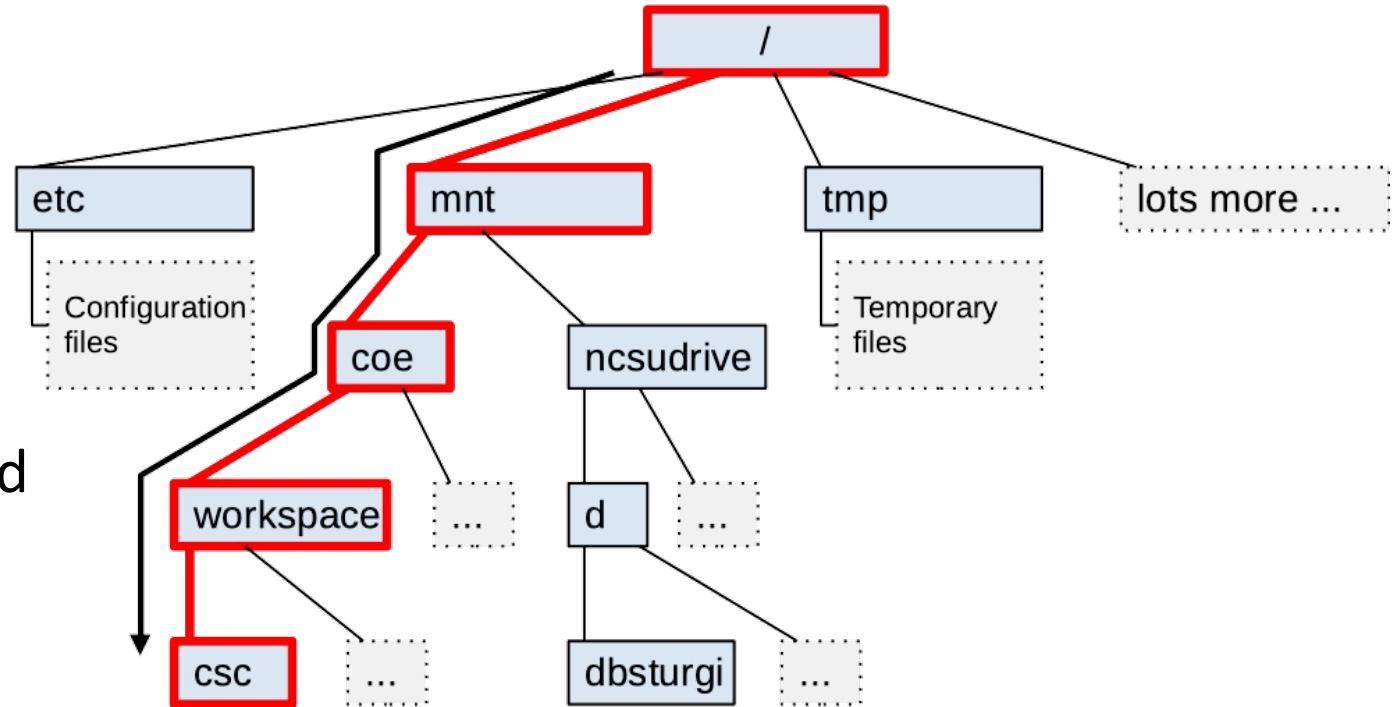


```
$ cd /mnt/coe/workspace/csc
$ pwd
/mnt/coe/workspace/csc
```

We could also have said:
`cd ../../../../coe/workspace/csc`

Tab completion

- The shell will complete complete file or directory names.
 - When you press **tab**.
 - Based on the letters you typed so far.
- On the previous example, I really typed



```
$ cd /mnt/c tab w tab c tab
```

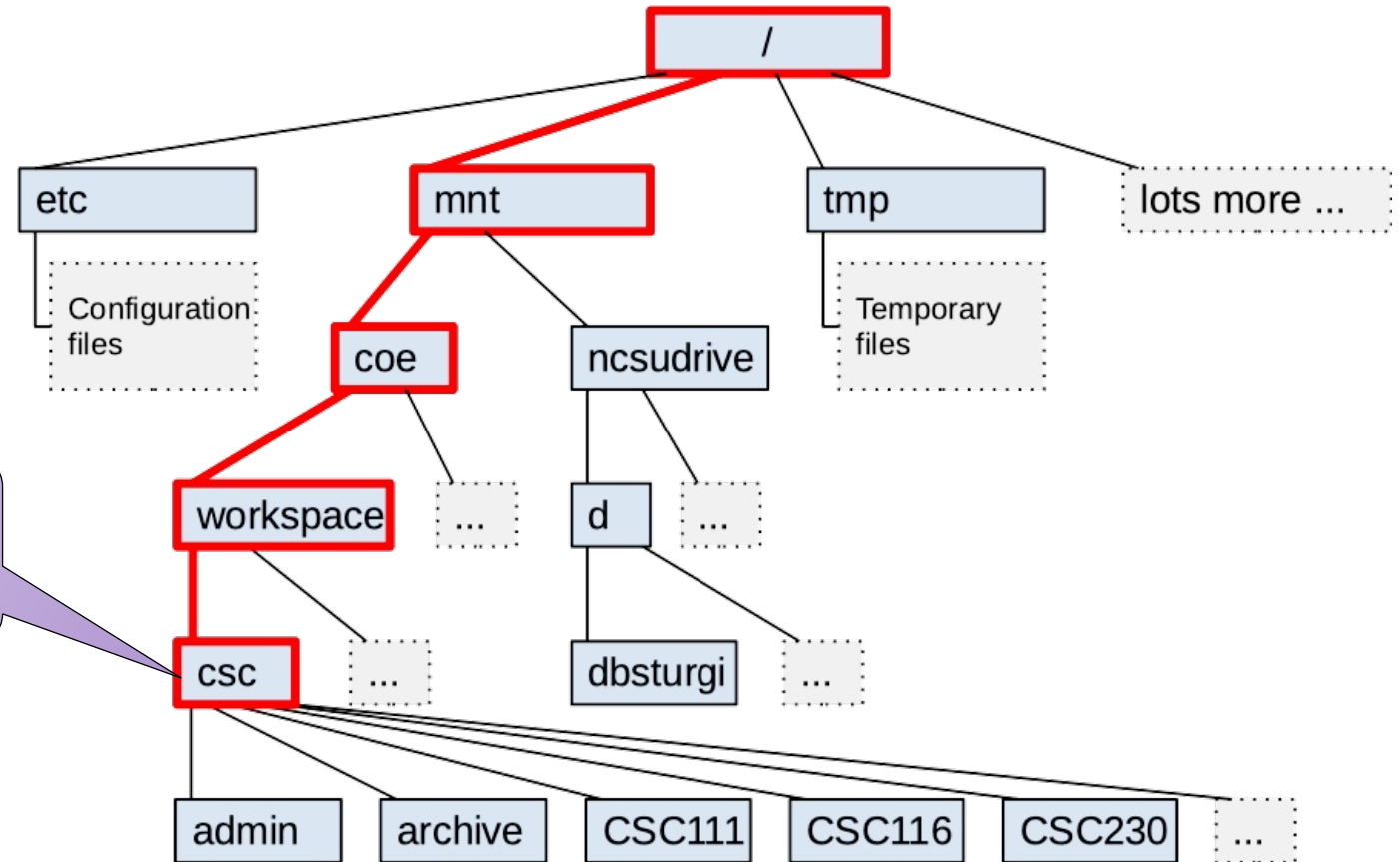
- As I typed, this expanded to

```
$ cd /mnt/coe/workspace/csc
```

ls : listing directory contents

- The **ls** command will list files in the current directory.

This is where we keep some files supporting CSC classes.



```
$ ls
```

```
admin  archive  CSC111  CSC116  CSC230  CSC246  CSC295  CSC506-1
```

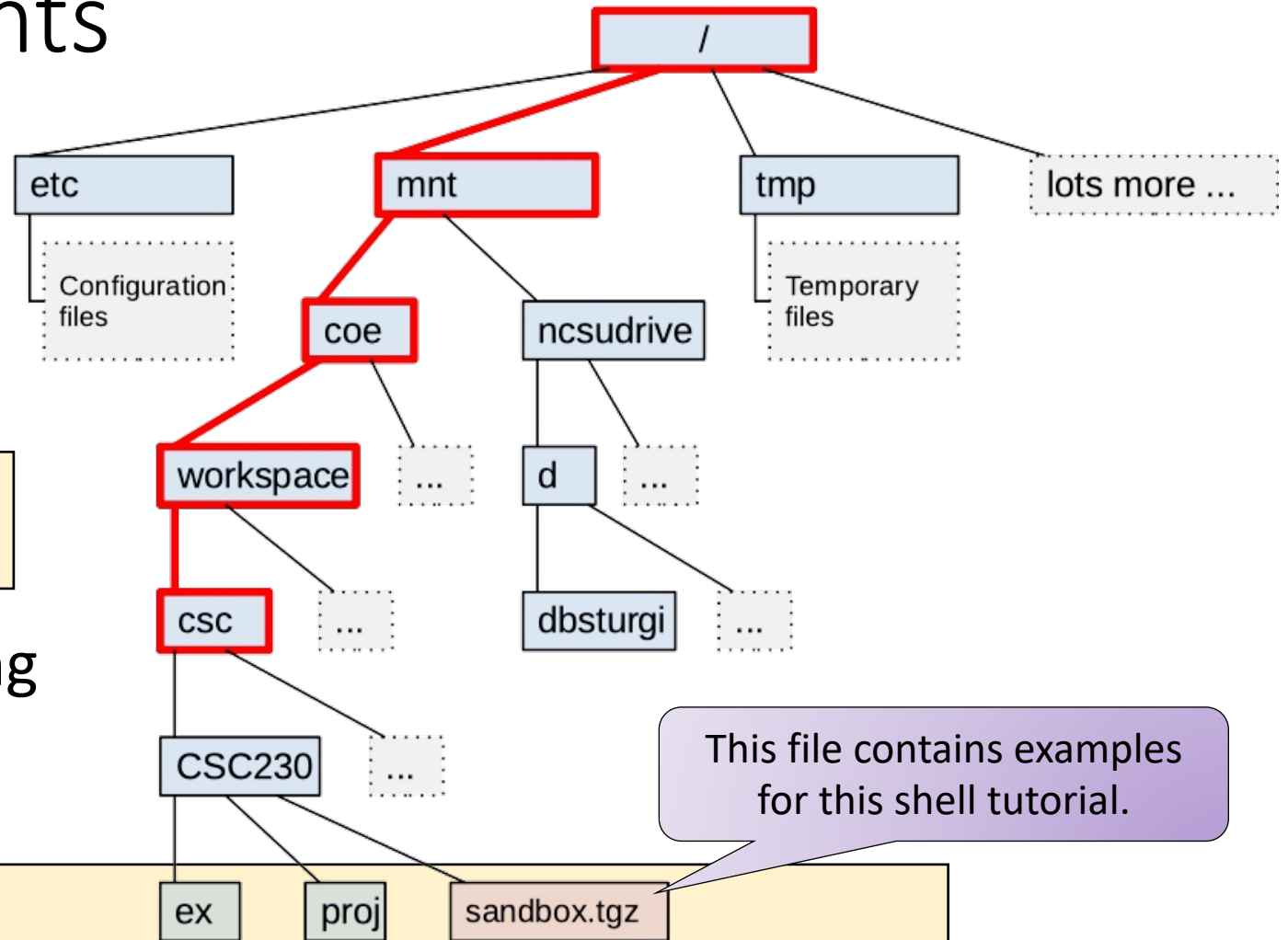
Command Arguments

- As an argument, **ls** can take a path to a directory as an argument.

```
$ ls CSC230
ex  proj  sandbox.tgz
```

- It can also take **-l** to get a long listing.
 - This is called an **option**.

```
$ ls -l CSC230
total 122
drwxrwx---+ 26 dbsturgi NCSU  672 Jan  2 10:20 ex
drwxrwx---+  3 dbsturgi NCSU   20 Jan  2 10:20 proj
-rwxrwx---+  1 dbsturgi NCSU 4052 Jan 15 16:59 sandbox.tgz
```



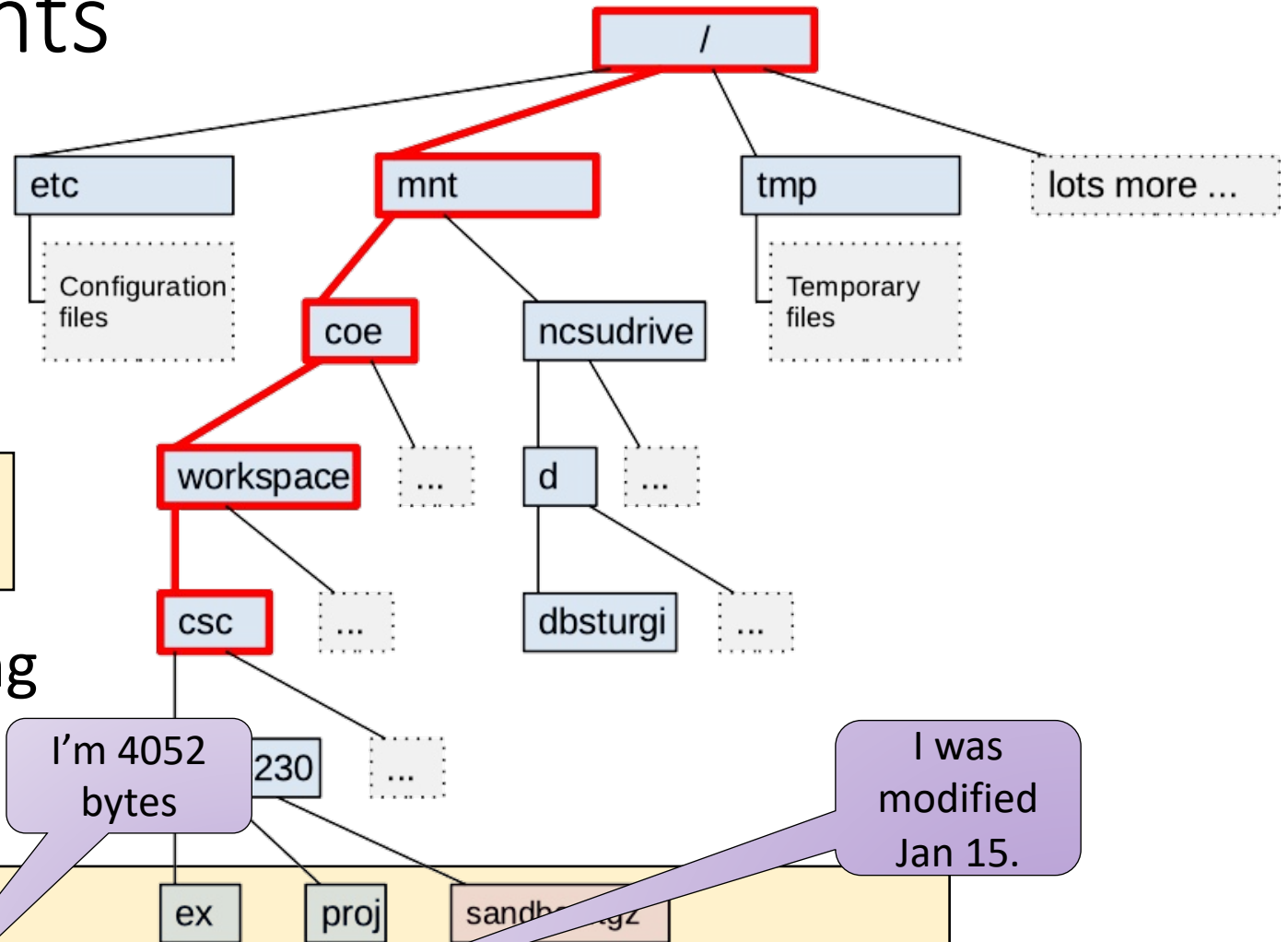
Command Arguments

- As an argument, **ls** can take a path to a directory as an argument.

```
$ ls CSC230
ex  proj  sandbox.tgz
```

- It can also take an option to get a long listing.
- This is called an **option**.

```
$ ls -l CSC230
total 122
drwxrwx---+ 26 dbsturgi NCSU 672 Jan  2 10:20 ex
drwxrwx---+  3 dbsturgi NCSU 20 Jan  2 10:20 proj
-rwxrwx---+  1 dbsturgi NCSU 4052 Jan 15 16:59 sandbox.tgz
```

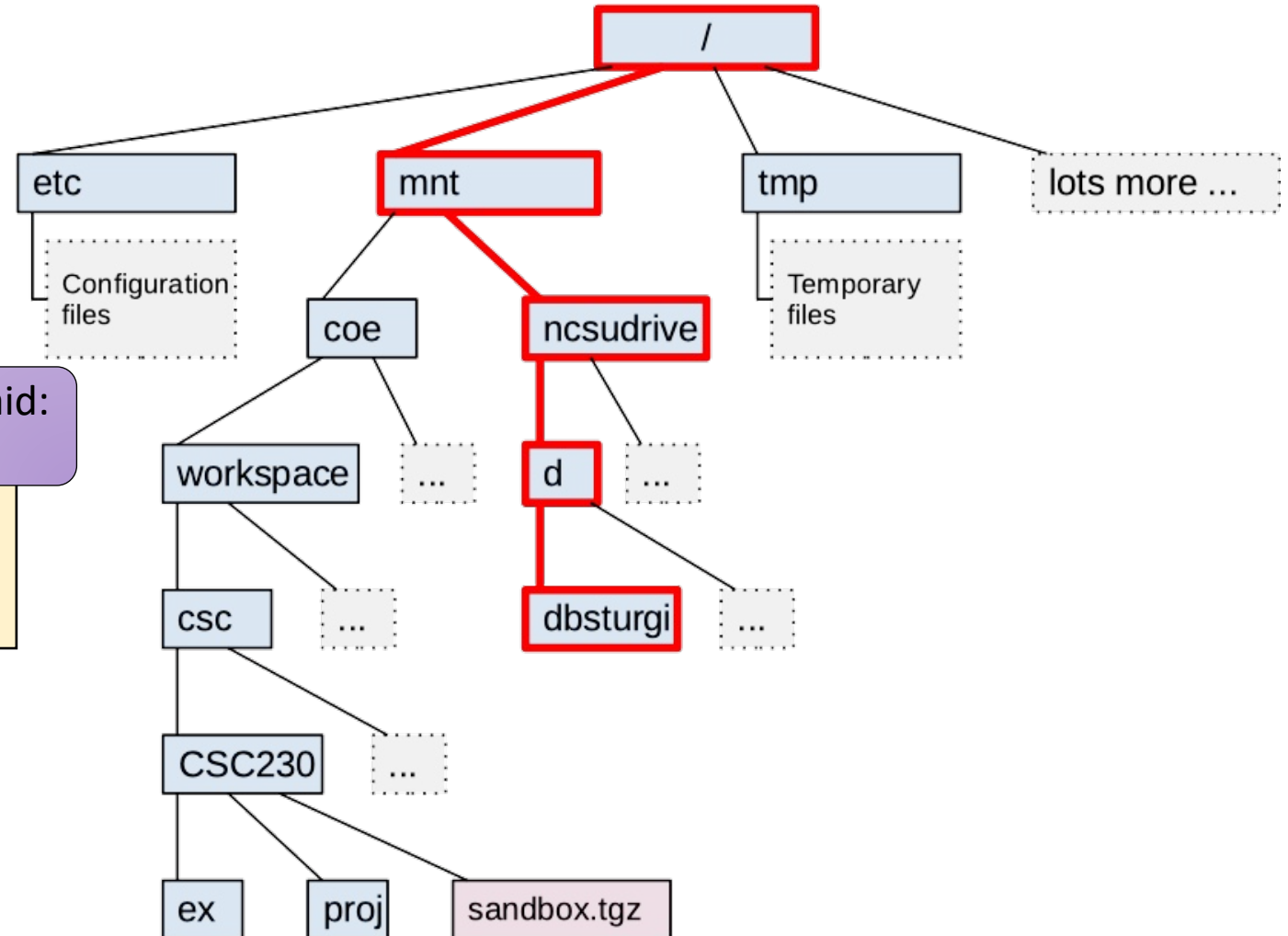


Returning Home

- Without any arguments, `cd` will take you back to your home directory.

```
$ cd  
$ pwd  
/mnt/ncsudrive/d/dbsturgi
```

We could also have said:
`cd ~`



In -s : Making shortcut (a symbolic link)

- It's a long path to CSC230
- We can use In -s to make a symbolic link to there in our home directory.

```
$ ln -s /mnt/coe/workspace/csc/CSC230 shared
```

- Then, this shortcut lets us get there via a shorter path.

```
$ cd shared
```

```
$ ls
```

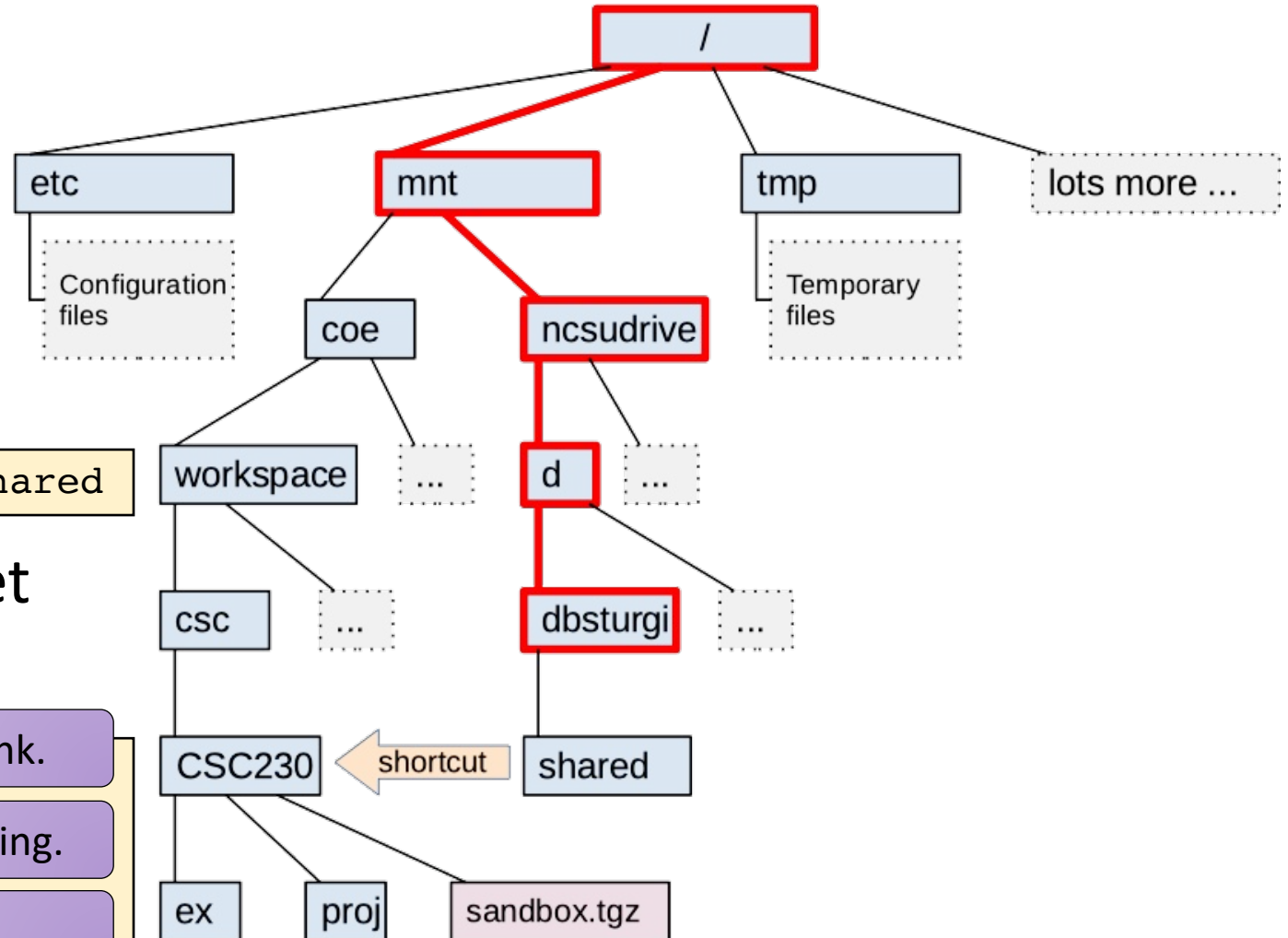
```
ex proj sandbox.tgz
```

```
$ cd ..
```

Follow the new link.

Get a directory listing.

Go back home.



cp : copy a file or directory

- The **cp** command lets you copy a file or directory.

- To copy just one file:

```
$ cp srcFile dest
```

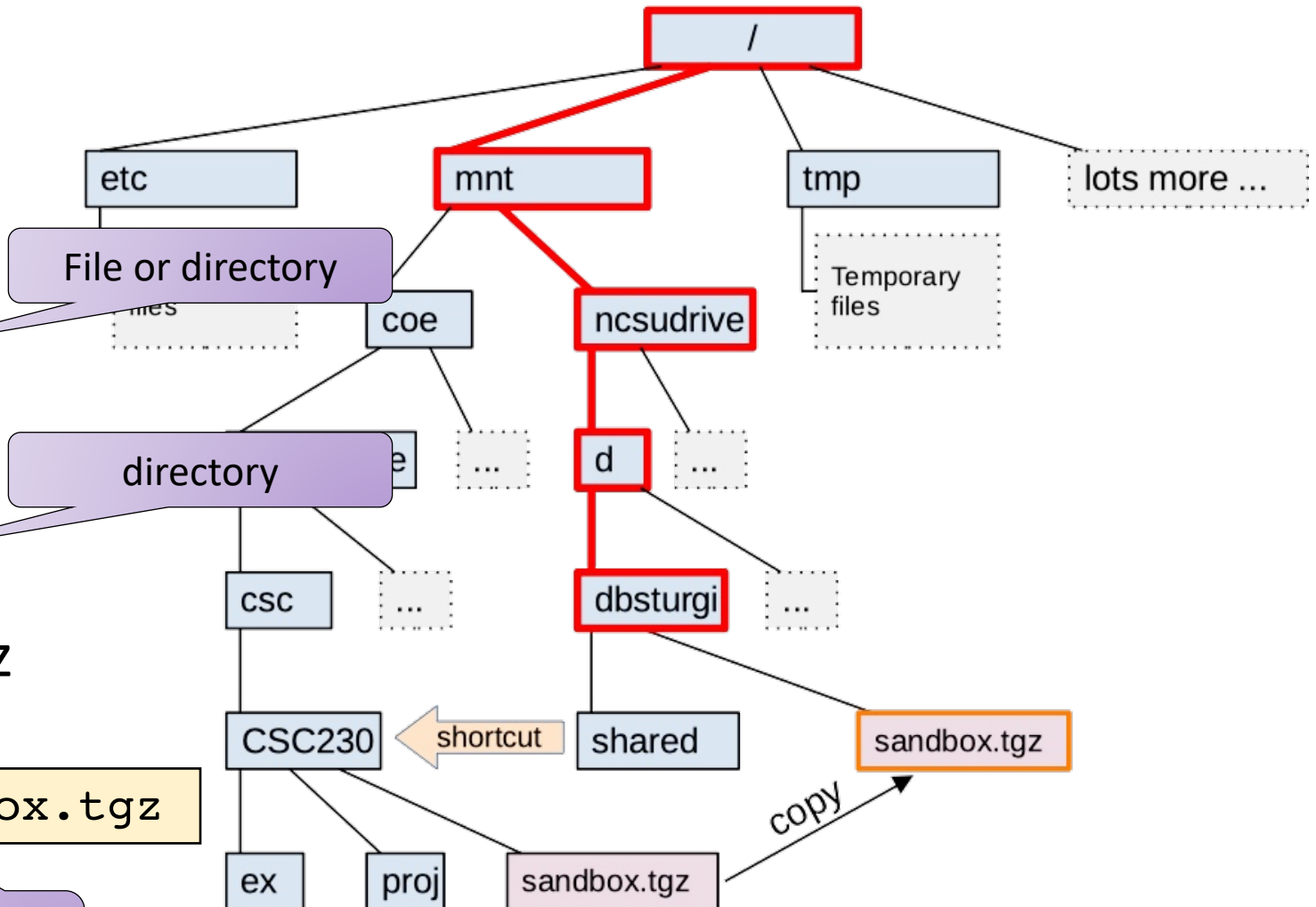
- With -r, recursively copy a directory and its contents.

```
$ cp -r srcDir dest
```

- Let's copy the sandbox.tgz file.

```
$ cp shared/sandbox.tgz sandbox.tgz
```

We could also have said:
cp shared/sandbox.tgz .



tar : create / extract from an archive

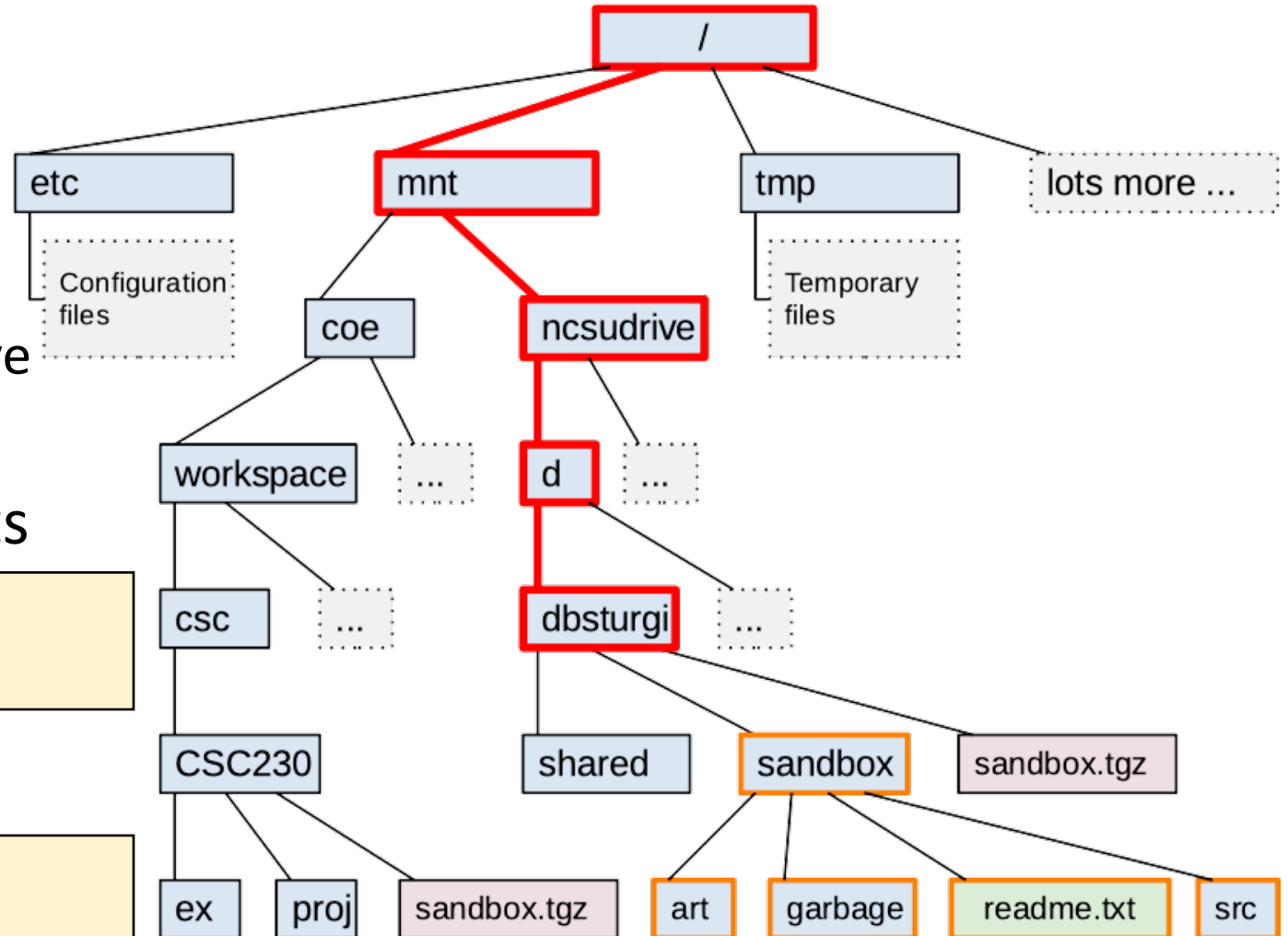
- Tar is a unix (tape) archive format.
 - The .tgz extension says this file is a compressed tar archive
- The **tar** command can give a listing of the archive contents

```
$ tar tzf sandbox.tgz  
... files in archive ...
```

- It can extract all the files.

```
$ tar xzf sandbox.tgz  
$ ls sandbox  
art  garbage  readme.txt  src
```

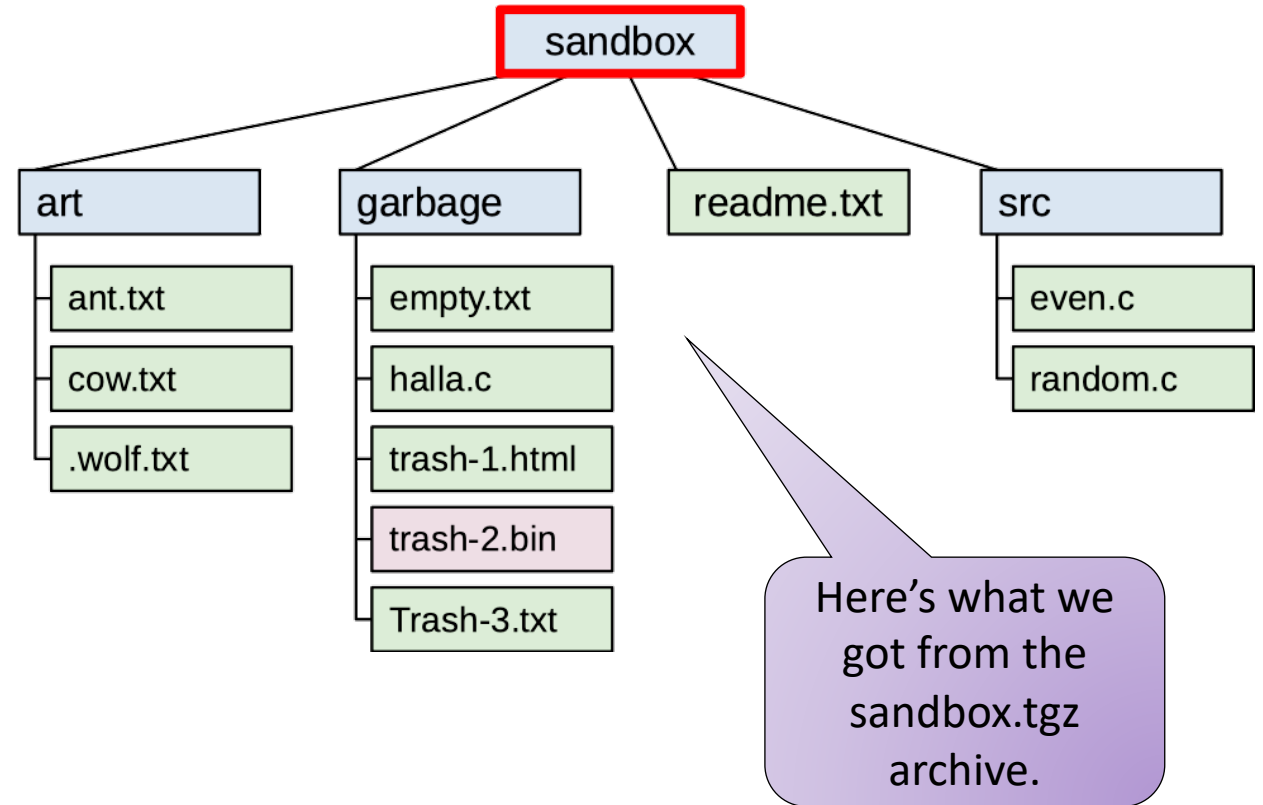
See what we got.



Playing in the sandbox

- The files in the sandbox are for trying out shell commands.
- We can't change the shared copy of these files under `/mnt/coe/workspace/csc`
- ... but this copy is ours.
- Let's have a look around.
- **ls -R** will show all files recursively

```
$ cd sandbox
$ ls -R
... all the files under sandbox ...
```

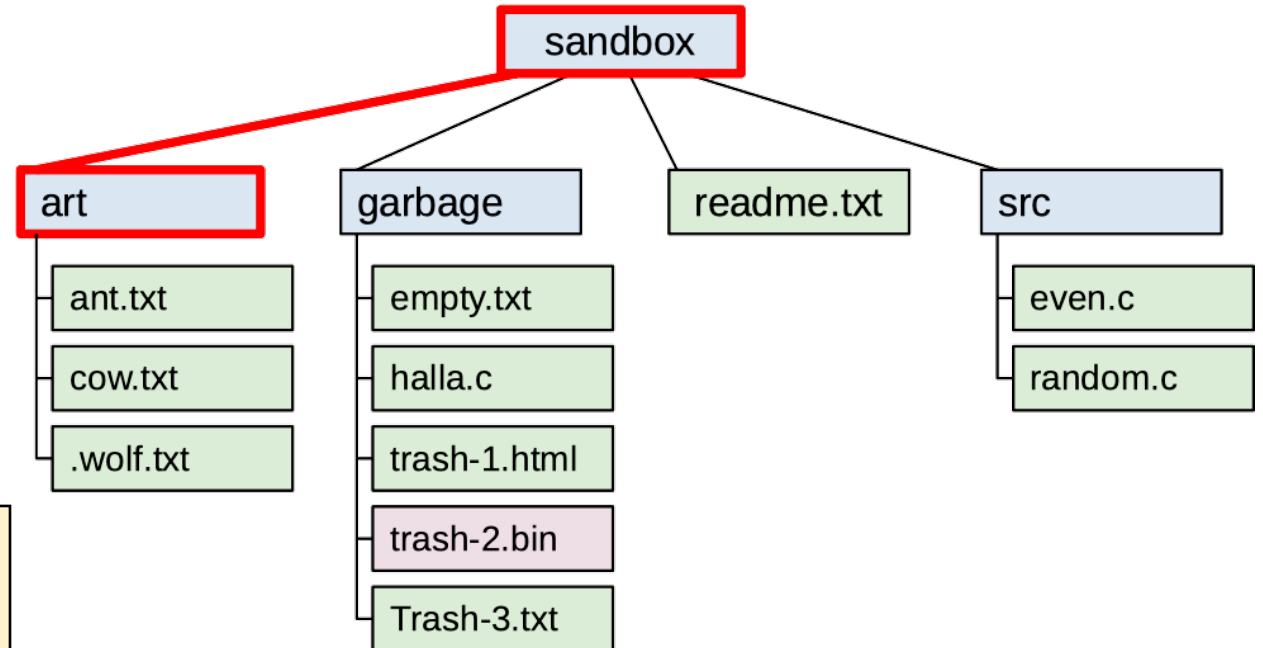


cat : view (or concatenate) files

- The art directory contains some ASCII art.
- We can use the cat program as a simple way to show the contents of a file in the terminal.

```
$ cd art
$ ls
ant.txt  cow.txt
$ cat cow.txt

      _n_n_
     /-----\00/-'
    /  ##  ## (oo)
   /  \##  __  ./
      |//YY \||/
snd  |||    |||
```



Hidden files

- Notice, we can't see .wolf.txt

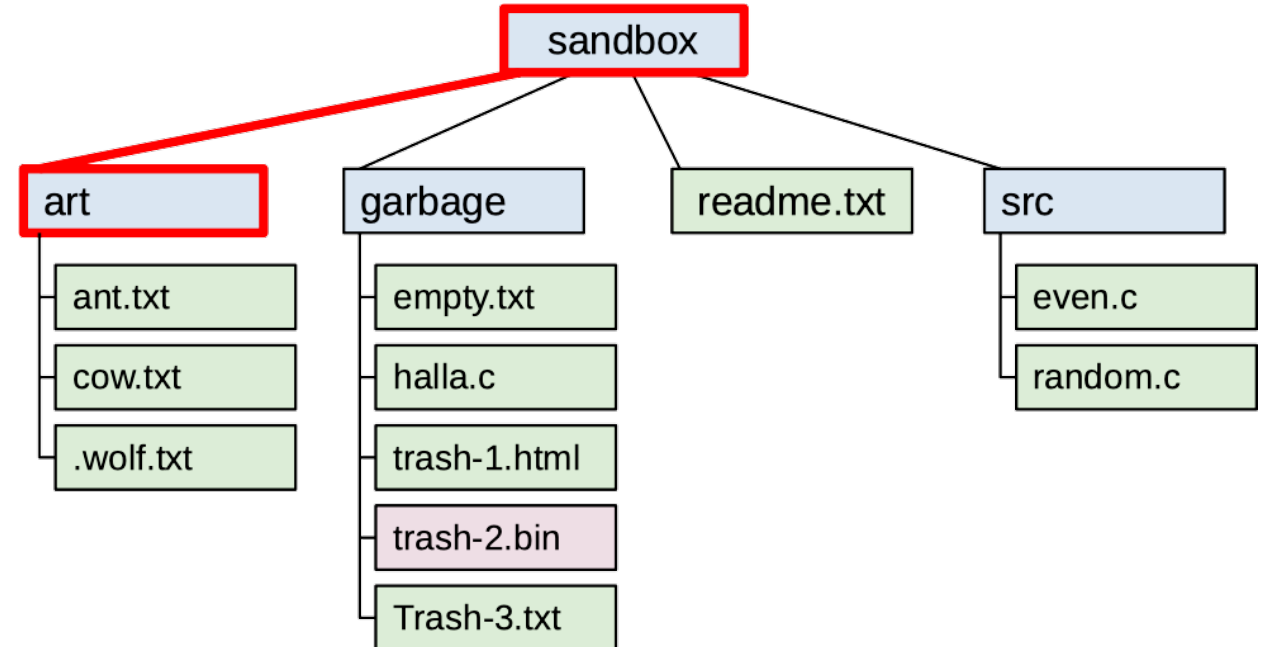
```
$ ls  
ant.txt  cow.txt
```

- Files starting with dot are considered **hidden**.
- The **-a** option will show hidden files.

```
$ ls -a  
.  ..  ant.txt  cow.txt  .wolf.txt
```

- You can combine options to ls.

```
$ ls -la  
... long listing of all files ...
```

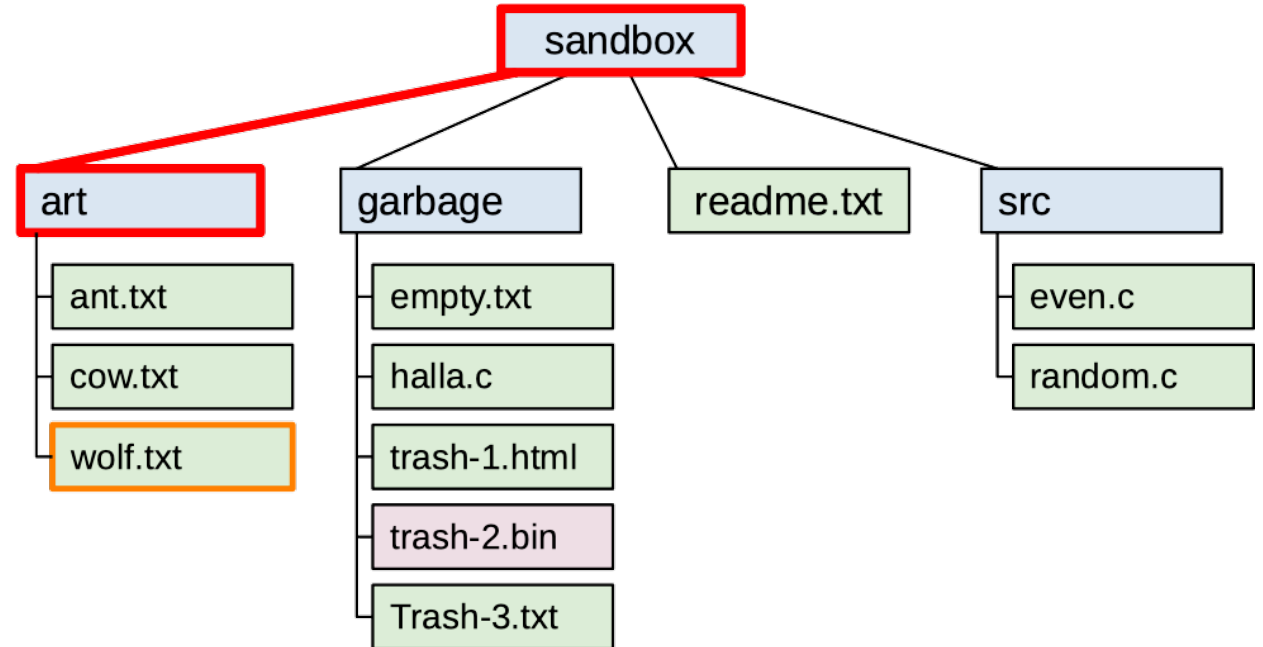


mv : move or rename files

- The **mv** command can move or rename a file or directory

```
$ mv .wolf.txt wolf.txt  
$ ls  
ant.txt  cow.txt  wolf.txt
```

Now we can see
it in a directory
listing.



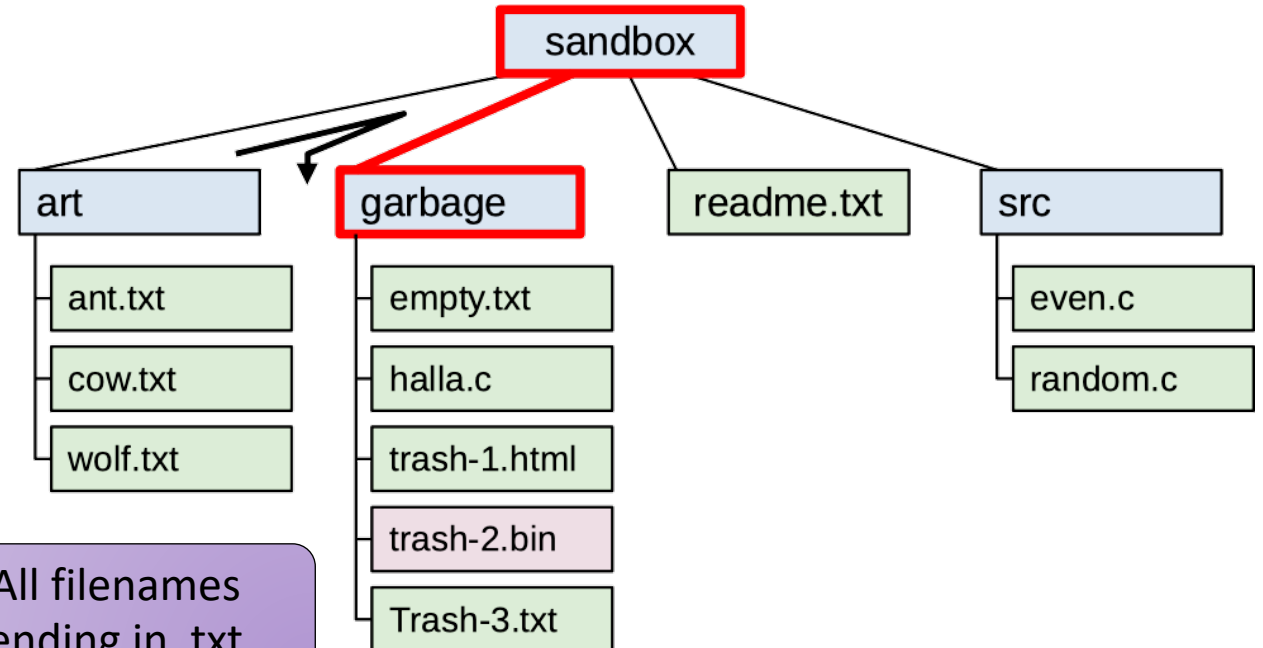
Pattern Matching

- The shell has a built-in pattern language for matching filenames and other strings.
- An asterisk will match any sequence of characters.

```
$ cd ../garbage
$ ls *.txt
empty.txt  Trash-3.txt
$ ls t*
trash-1.html  trash-2.bin
```

All filenames
ending in .txt

All filenames
starting with t.



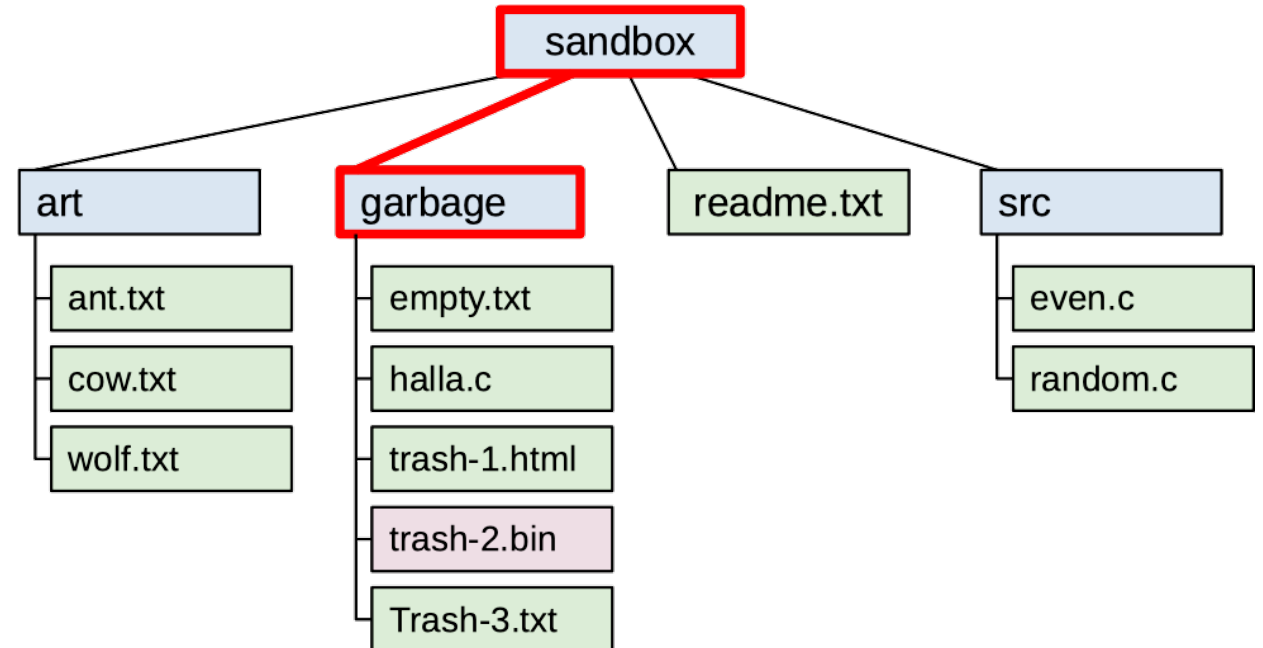
Pattern Matching

- You can use square brackets to match individual characters in a set.

```
$ ls [tT]*  
trash-1.html  trash-2.bin  Trash-3.txt
```

- Or, question mark will match any single character.

```
$ ls *-?.*  
trash-1.html  trash-2.bin  Trash-3.txt  
$ ls *.???  
empty.txt  trash-2.bin  Trash-3.txt
```



less : paginating and searching in a text file

- We can use cat see what's in empty.txt and Trash-3.txt

```
$ cat empty.txt
```

```
$ cat Trash-3.txt
```

```
This is a text file, but it  
can be deleted.
```

Nothing printed.

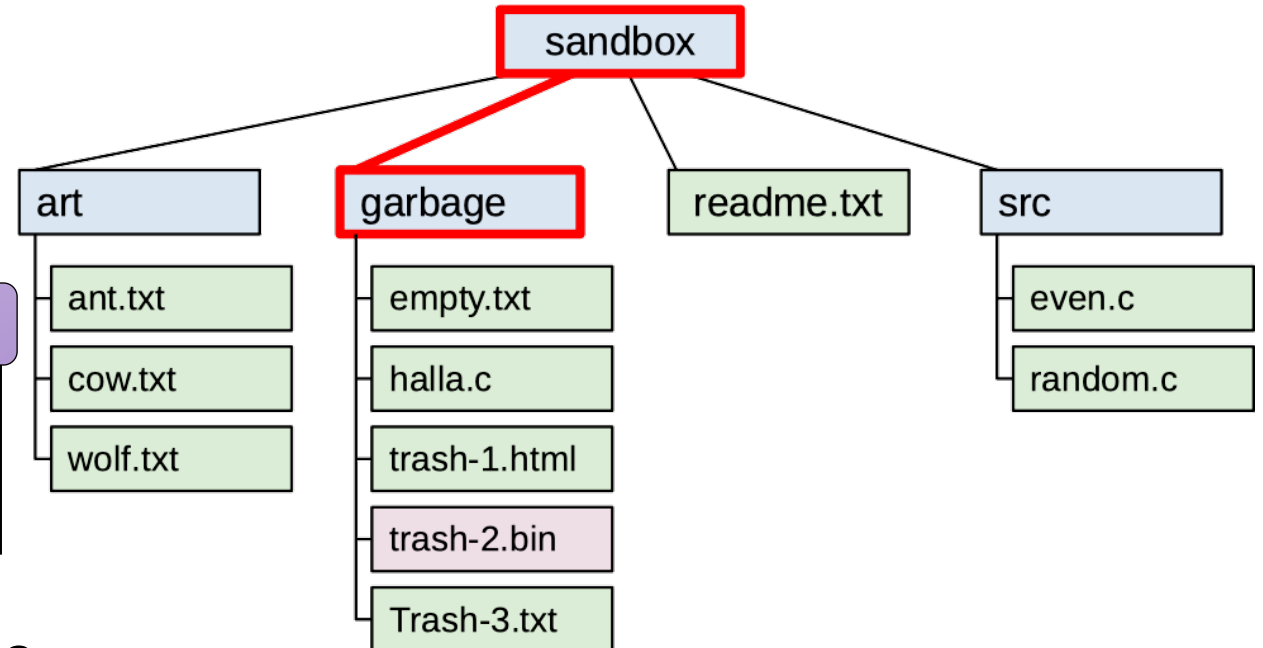
- Trash-1.html is too long to see the whole file at once.

```
$ cat trash-1.html
```

```
... lots of output ...
```

```
$ less trash-1.html
```

Viewing the file in less.
Arrows to move up and down.
/ to search for a string
q to quit.

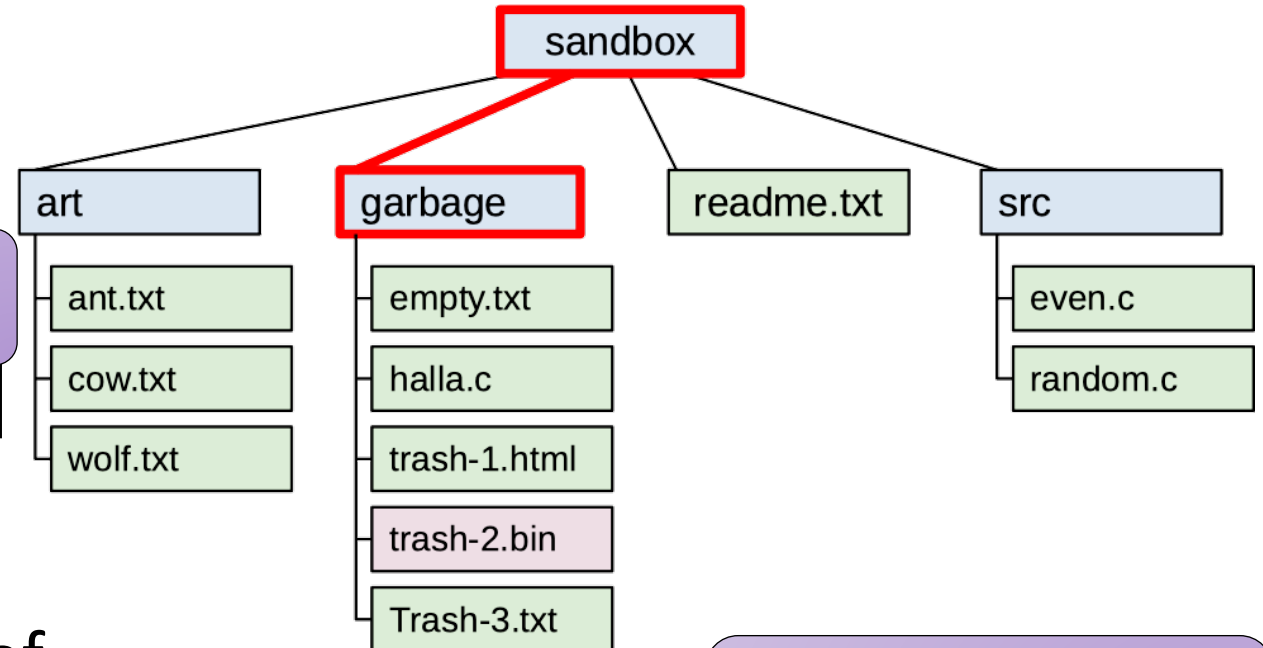


hexdump : viewing a binary file.

- A binary file contains non-text data.

```
$ cat trash-2.bin  
j?*_.XfD..T...%.  
.....p.....b.>
```

Looks like
garbage



- hexdump -C will show the value of each byte.

```
$ hexdump -C trash-2.bin
```

00000000	6a	3f	2a	5f	e9	58	66	44	c6	ff	54	9b	9f	04	25	f0	j?*_.XfD..T...%.
00000010	88	1c	f1	94	b4	e0	70	a0	b7	b3	09	19	82	62	16	3e	p.....b.>
00000020																	

Lots of these aren't valid
ASCII codes.

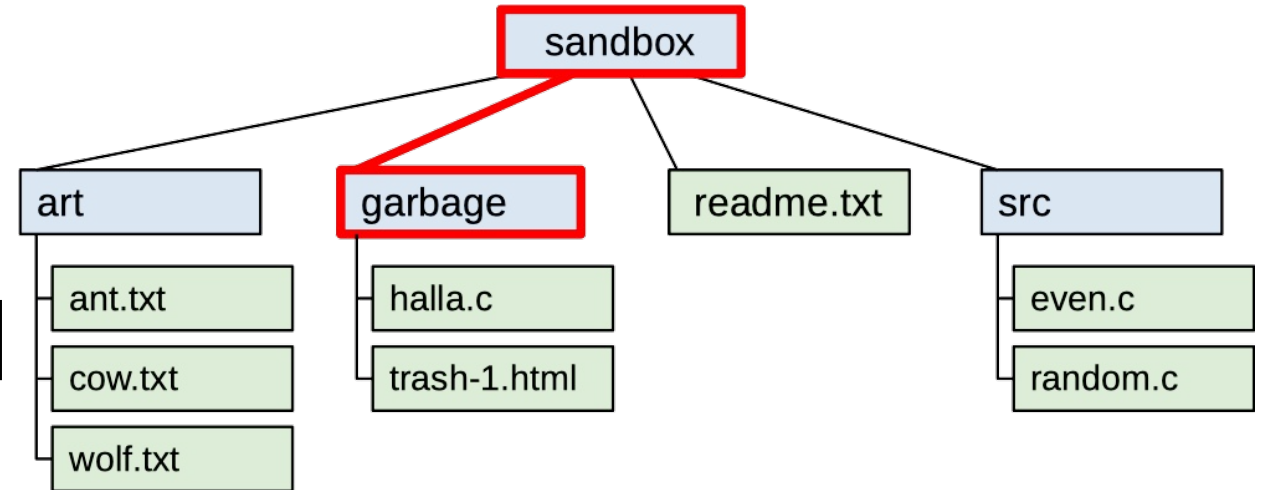
rm : removing files

- We can delete a single file with **rm**.

```
$ rm trash-2.bin
```

- We can use a shell pattern to delete multiple files (be careful)

```
$ rm *.txt  
$ ls  
halla.c  trash-1.html
```



See. Three files removed.

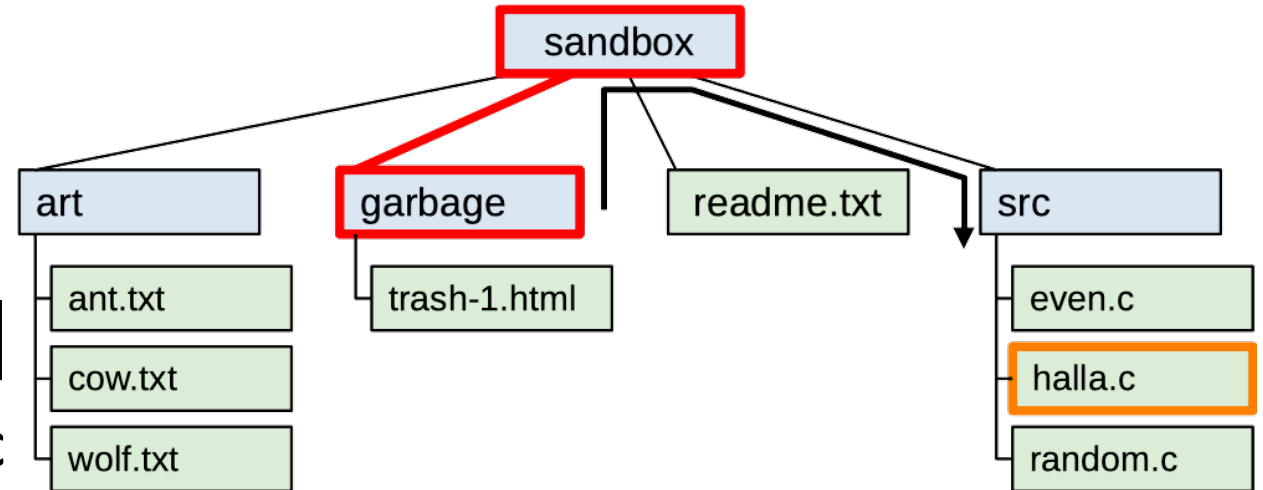
mv : moving a file

- The halla.c file looks like a C program.

```
$ less halla.c
```

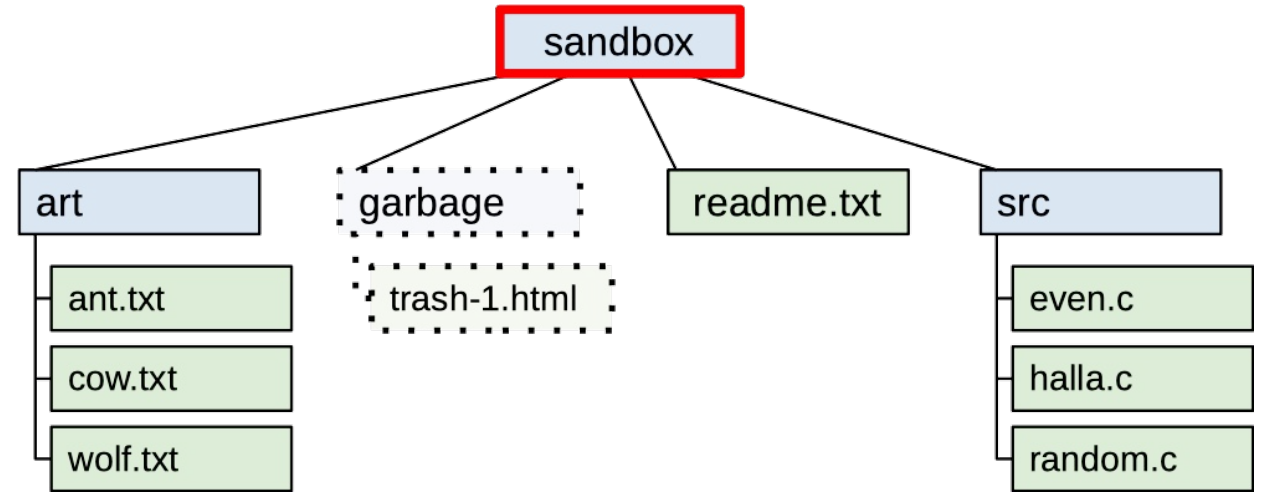
- Let's use **mv** to move it to the src directory.

```
$ mv halla.c ../src
```



rm -r : recursive remove

- I'd say the garbage directory and everything in it can be deleted.
- By default, rm won't remove directories.



```
$ cd ..  
$ rm garbage  
rm: cannot remove 'garbage': Is a directory
```

- With the -r option, it will perform a recursive remove (be careful)

```
$ rm -r garbage  
$ ls  
art  readme.txt  src
```

See, the garbage directory is gone.

Your command history

- In the src directory, we can use gcc with the regular options to build the programs.

```
$ gcc -Wall -std=c99 even.c -o even
$ gcc -Wall -std=c99 halla.c -o halla
$ gcc -Wall -std=c99 random.c -o random
```

- This creates executable files for each.
- We can run one by giving a relative path to the executable in the current directory.

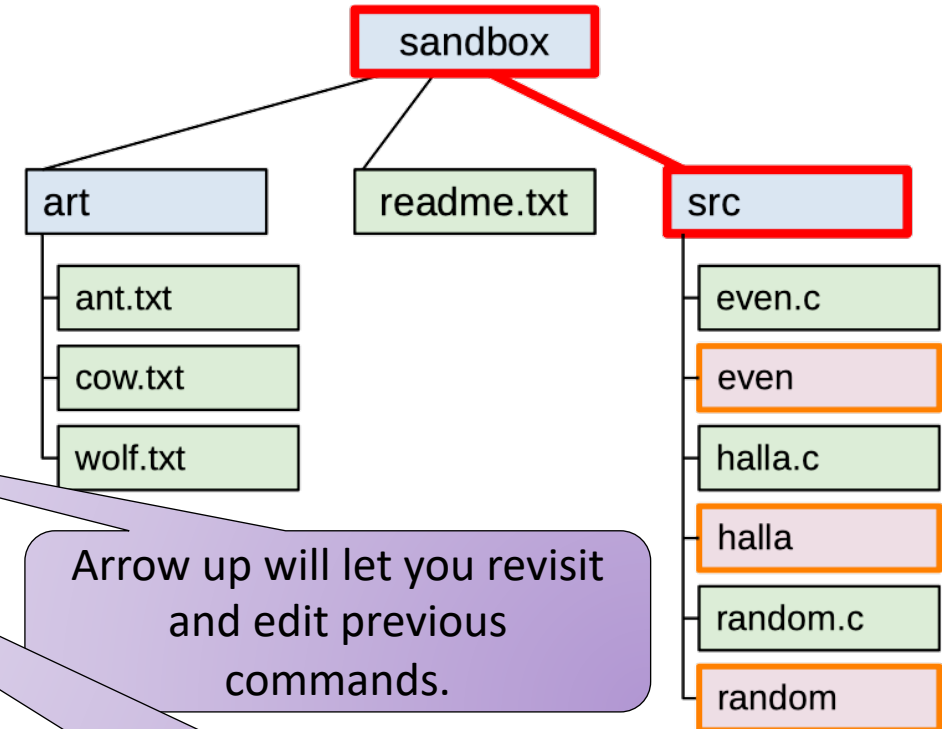
```
$ ./halla
Hej världen.$
```

Hello world, in Swedish.

Why is my next prompt over here?

Arrow up will let you revisit and edit previous commands.

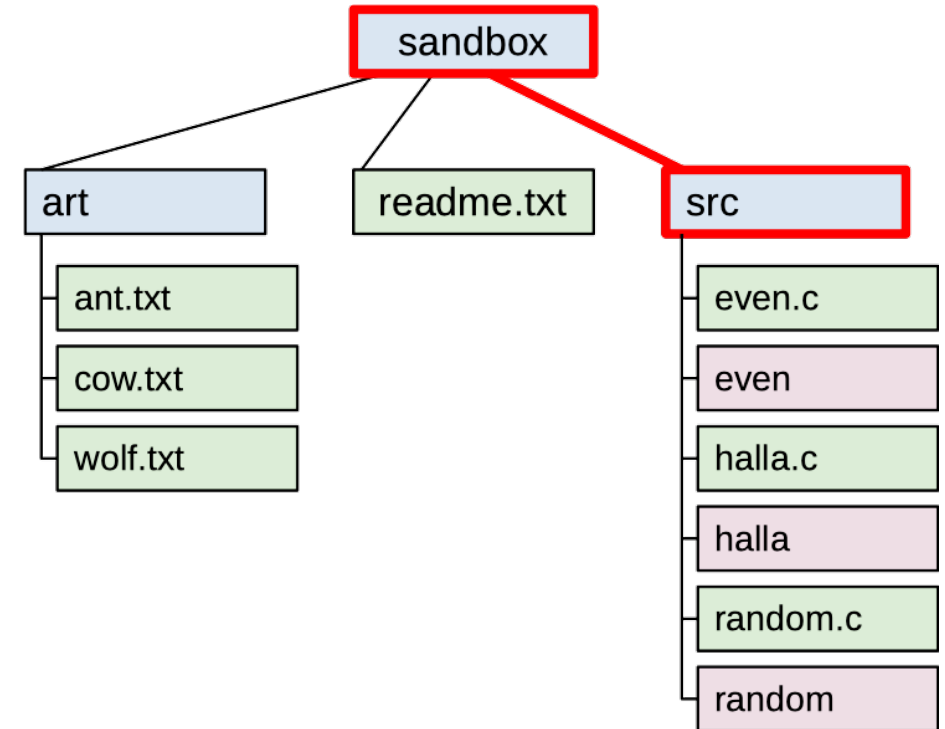
Ctrl-R will let you reverse search through your command history.



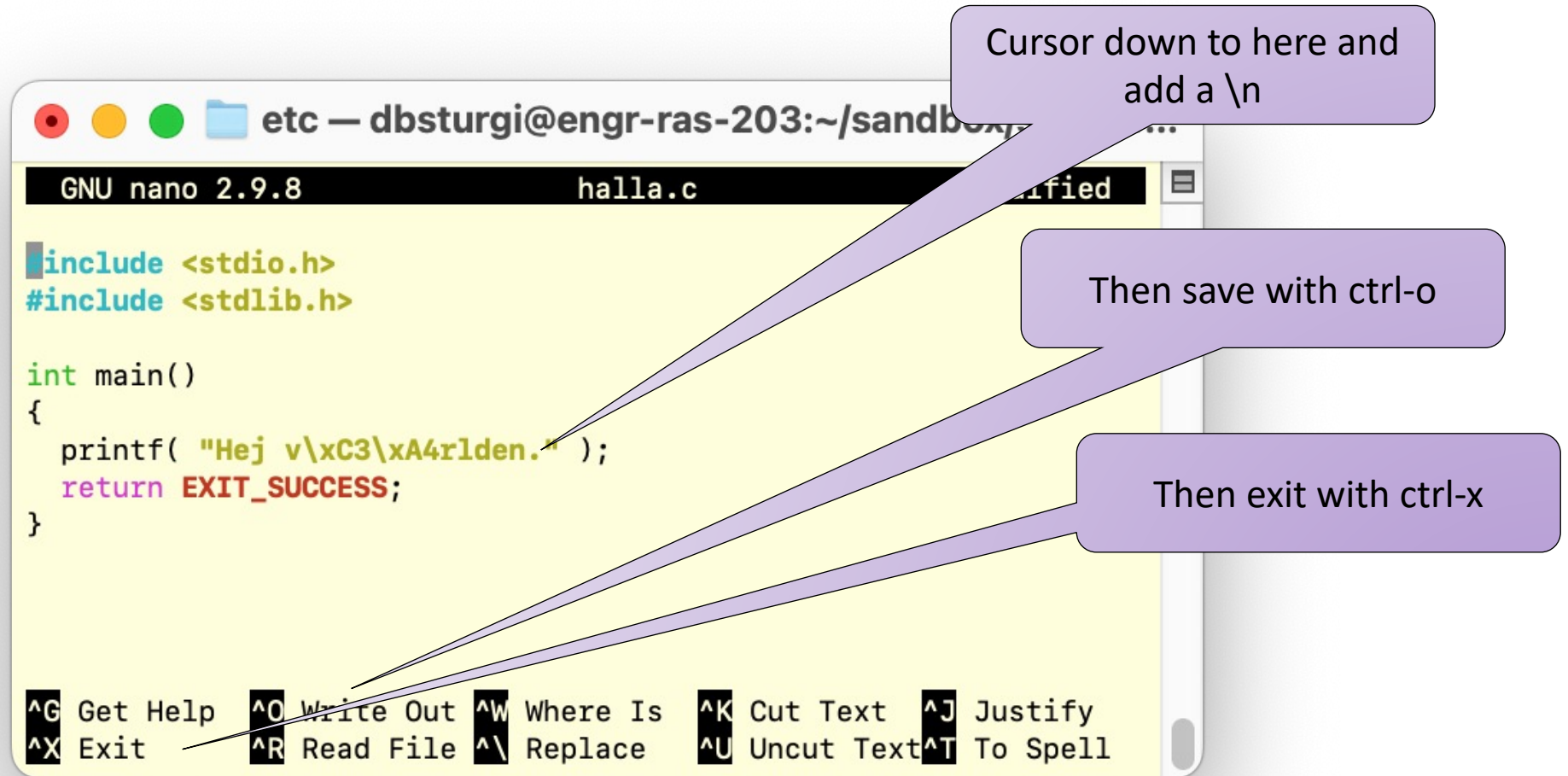
Making a quick program edit

- You'll want to use a good editor to write most of your programs
- ... but, sometimes it's nice to be able to make a quick edit right in the terminal.
- Programs like nano (or vim or emacs) can let you do that.
- Let's add a newline to the end our output.

```
$ nano halla.c
```



Making a quick program edit



Re-building and re-running

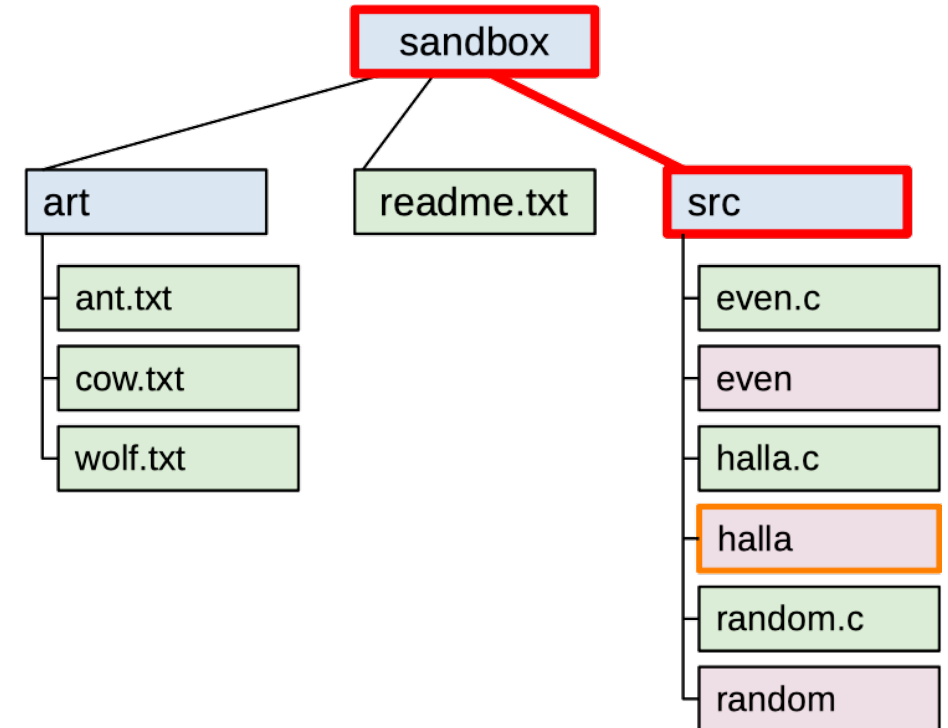
- We can rebuild the program.

```
$ gcc -Wall -std=c99 halla.c -o hella
```

- And try it a gain.

```
$ ./halla  
Hej världen.  
$
```

Good. It looks like the
line termination is fixed.



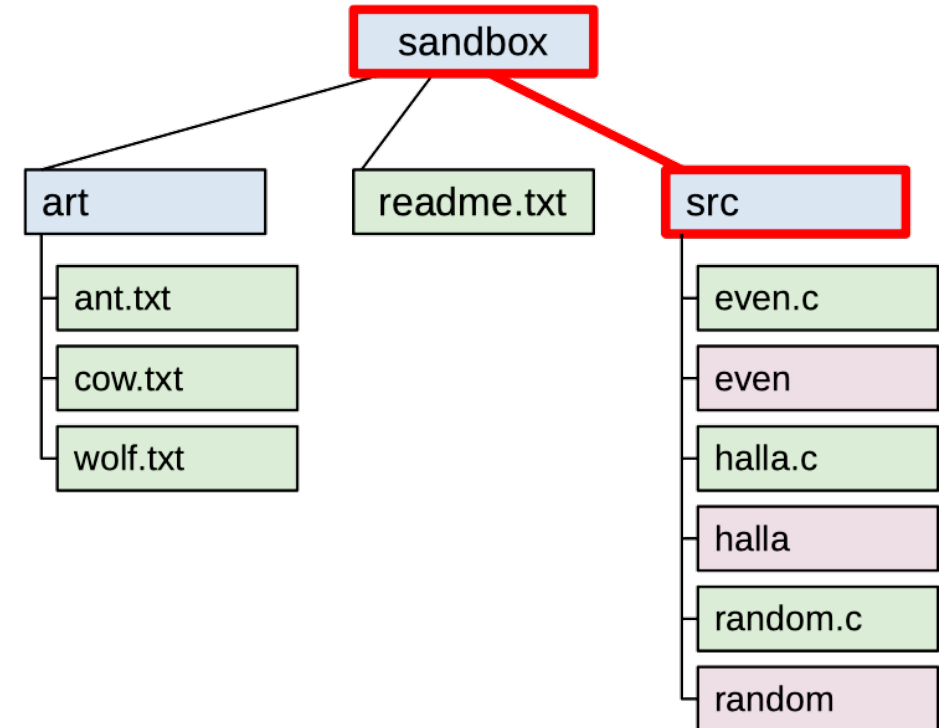
Environment variables and echo

- The shell has a syntax for using **environment variables**.
- These let you store strings to use in shell commands.
- You can set a variable with syntax like:

```
$ name=bart
```

- The syntax **\$p** expand to the value in **p**
- The echo command just prints out its arguments.
 - It's gives us an easy way to check the value of a variable.

```
$ echo Hello $name  
Hello bart
```



Shell programming

- Using an environment variable can save you a little repetition in a command

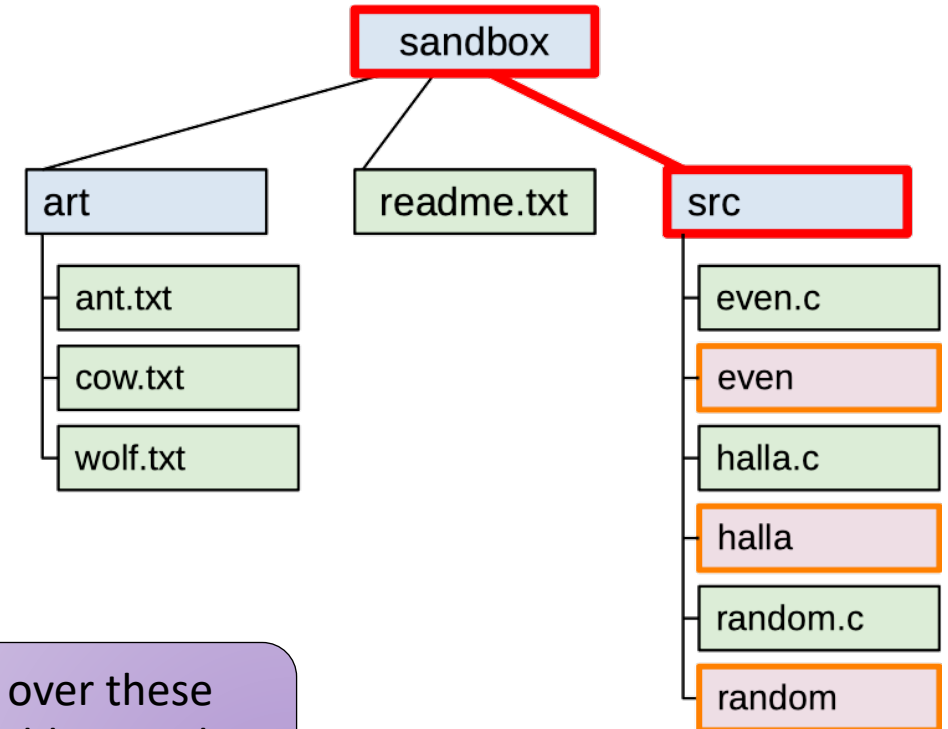
```
$ p=halla
$ gcc -Wall -std=c99 $p.c -o $p
```

- More importantly, it can let you write things like loops in shell.

```
$ for p in even halla random
> do
> gcc -Wall -std=c99 $p.c -o $p
> done
```

This will iterate over these values, giving variable, p, value on one iteration.

This shell prompt means you're not done entering a command.



Shell scripts

- You can put a sequence of shell commands in a text file.
- Then, you can run it to execute those commands any time you want.
- This is called a **shell script**.
- The test scripts we provide with most projects work like this.

This says run me with bash.

```
#!/bin/bash

for p in even halla random
do
    gcc -Wall -std=c99 $p.c -o $p
done
```

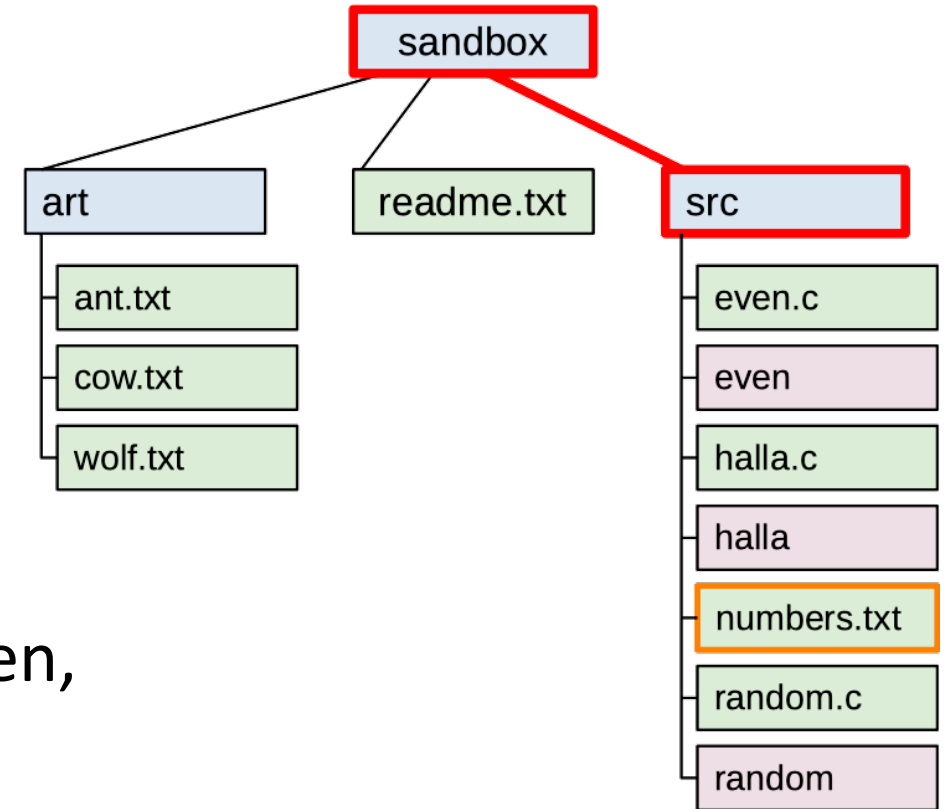
Standard Output

- The random.c program prints ten random numbers to the terminal (standard output)

```
$ ./random
84
... numbers omitted ...
22
```

- Instead of letting this output go to the screen, you can send it to a file instead.
- This is called **output redirection**.

```
$ ./random > numbers.txt
$ less numbers.txt
```



Standard Input

- The even.c program reads numbers from the user (standard input) and reports if they're even or odd.

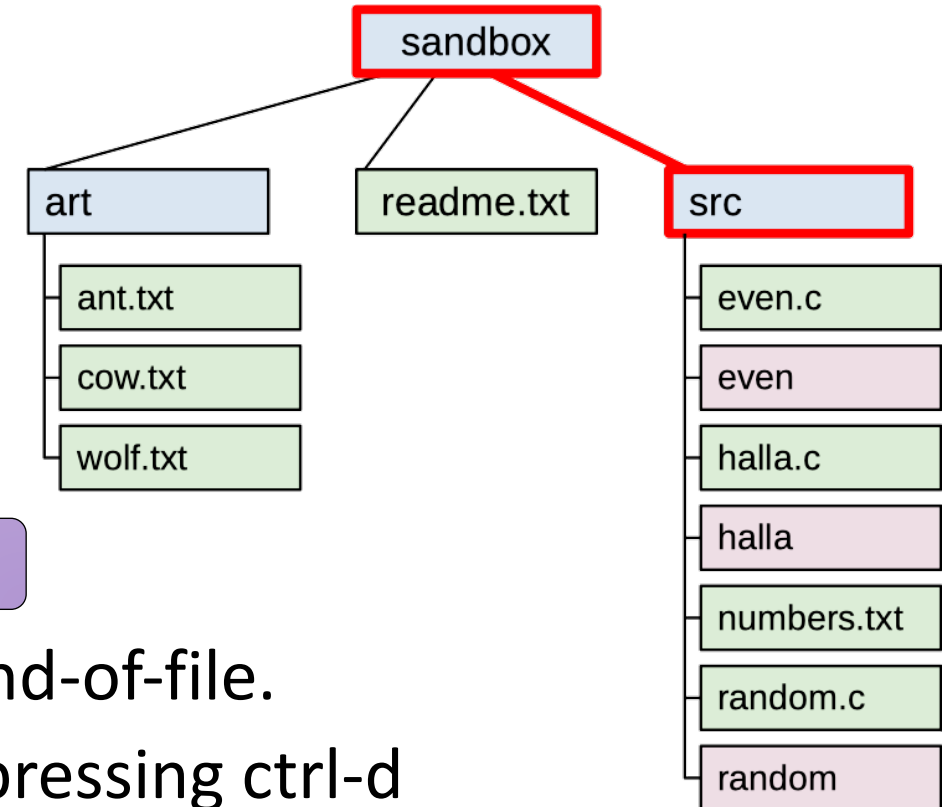
```
$ ./even
27
27 Odd
42
42 Even
blork
```

Can't parse as an integer.

- This program will stop on invalid input or end-of-file.
- You can signal the end-of-file condition by pressing ctrl-d

```
$ ./even
-5
-5 Odd
$
```

I pressed ctrl-d here.



Input redirection and pipes

- We can redirect standard input, so even.c gets input from a file when it tries to read from the user.

```
$ ./even < numbers.txt
```

```
84 Even
```

```
... more output ...
```

```
22 Even
```

```
$ ./even < numbers.txt > results.txt
```

Gets EOF when we reach the end of this file.

We can direct input and output at the same time.

- We can send the output of one program directly as input to another.
- That's done with a pipe (vertical bar)

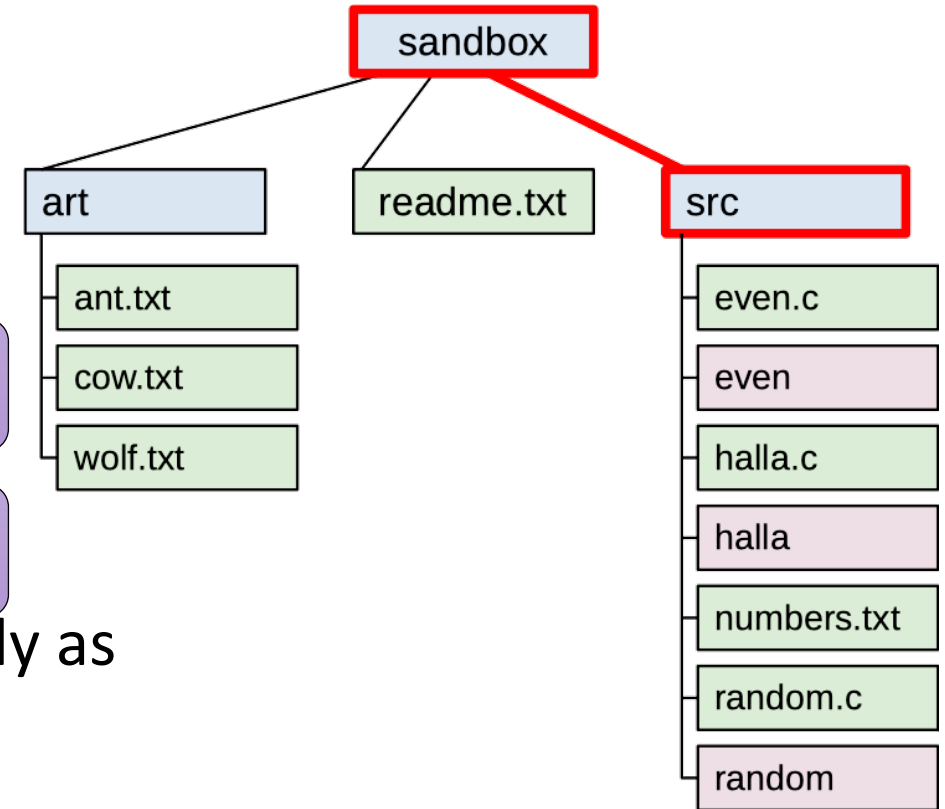
```
$ ./random | ./even
```

```
84 Even
```

```
... more output ...
```

```
22 Even
```

Both programs are run at the same time.



mkdir : making a subdirectory

- You can use **mkdir** to make directories and organize your files.

```
$ mkdir prog1  
$ mkdir prog2 prog3
```

You can make multiple subdirectories at once.

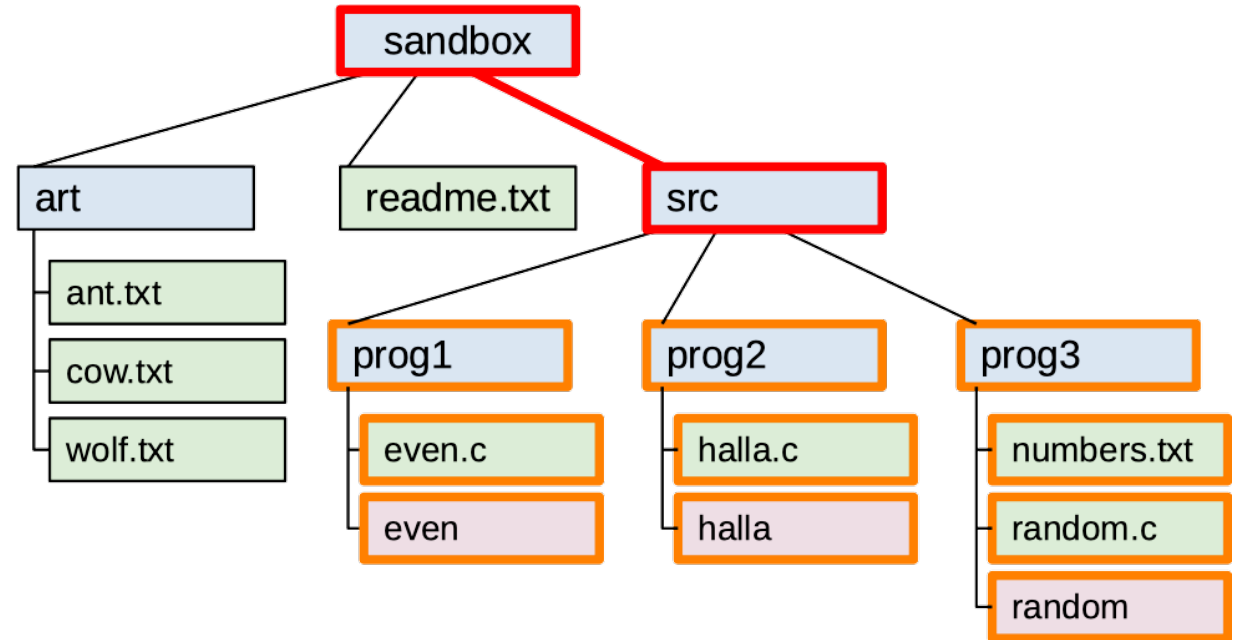
- Let's move each program into its own subdirectory.

```
$ mv even even.c prog1  
$ mv halla* prog2  
$ mv random* numbers.txt prog3  
$ ls -R  
... recursive directory listing ...
```

You can list multiple files to move.

You can use a pattern to name multiple files.

You can combine patterns and explicit filenames.



Cleaning up

- Once we're done, we can clean up the sandbox files

```
$ cd ../..
$ rm -r sandbox
$ rm sandbox.tgz
$ rm shared
```

Back up to my home directory.

Remove files for this tutorial.

- You can exit your ssh login when you're done.

```
$ exit
```

You could keep this link, but you probably won't need it.

